

红豆的算法模板

竞赛编程算法与数据结构

ACM / ICPC 竞赛模板集锦

2026 年 10 月

编译于 October 9, 2026



Contents

1 竞赛框架 3
- 1.1 基础框架 3
- 1.2 对拍器/交互器/数据生成器 4
  - 1.2.1 对拍器 4
  - 1.2.2 数据生成器 5
  - 1.2.3 交互器 6

2 数学 7
- 2.1 结论 7
  - 2.1.1 组合数学 7
  - 2.1.2 位运算 7
  - 2.1.3 重排不等式 7
  - 2.1.4 数论 7
  - 2.1.5 卡特兰数 8
- 2.2 随机数生成器 8
- 2.3 位运算函数 8
  - 2.3.1 返回二进制位数 8
  - 2.3.2 返回二进制的高位/低位 8
- 2.4 取整 8
- 2.5 快速幂 $O(logX)$ 9
- 2.6 组合数学封装（组合 排列 阶乘 逆元 快速幂） 9
- 2.7 质数判定 10
- 2.8 快速预处理质因子 $O(NloglogN)$ 10
- 2.9 欧拉筛 10
  - 2.9.1 朴素欧拉筛 10
  - 2.9.2 SPF 欧拉筛（支持快速质因数分解 O(logn)，快速质数判断 O(1)) 10
- 2.10 最大公约数 最小公倍数 11
- 2.11 拓展欧几里得算法 12
- 2.12 乘法逆元 12
  - 2.12.1 费马小定理解（快速幂） 12
  - 2.12.2 拓展欧几里得解 $O(logX)$ 12
  - 2.12.3 （离线）线性递推解 $O(N)$ 12
- 2.13 异或线性基 12
- 2.14 矩阵 13
  - 2.14.1 矩阵封装 13

3 基础算法 15
- 3.1 最优化方法 15
  - 3.1.1 整数域 15
  - 3.1.2 实数域 15
- 3.2 前缀和与差分 16
- 3.3 高精度 16
- 3.4 离散化 17
- 3.5 排序 18
  - 3.5.1 归并排序 18
  - 3.5.2 快速排序 18
- 3.6 快速读入 18
- 3.7 回滚莫队 18

4 数据结构 20
- 4.1 单调队列 20

4.2 单调栈 20
4.3 树状数组 Fenwick BIT 20
4.3.1 半动态区间和（单点修改） 20
4.3.2 动态区间和（区间修改） 21
4.4 ST 表（Sparse Table） 21
4.5 线段树 22
4.5.1 ACL 线段树（好用，推荐，全能） 22
4.5.2 泛型线段树（自定义 Info，Merge，单点修改，区间查询，无 lazy） 26
4.5.3 WIDA 模板线段树 27
4.6 珂朵莉树（OD Tree） 28
4.7 主席树 29
4.7.1 单点加 区间查询 29
4.7.2 kth 主席树 31
4.8 串 32
4.8.1 Tire 字典树 32
4.8.2 KMP 32
4.8.3 AC 自动机 33
5 图论 34
5.1 知识补充 34
5.1.1 平面图 34
5.2 常见子问题 35
5.2.1 判断图的连通性 35
5.3 树论 35
5.3.1 树的直径 35
5.3.2 树的重心 36
5.3.3 最近公共祖先 LCA 36
5.3.4 树链剖分/轻重链剖分 39
5.4 并查集（DSU） 41
5.5 拓扑排序 42
5.6 最短路 42
5.6.1 单源最短路 42
5.6.2 多源最短路 44
5.6.3 DAG 中最长路 45
5.7 最小生成树 45
5.7.1 朴素 Prim $O(n^2)$ 45
5.7.2 稀疏图 堆优化 Prim $O(mlogn)$ 46
5.7.3 稠密图 kruskal $O(mlogm)$ 46
5.8 二分图 46
5.8.1 染色法判定二分图 46
5.8.2 二分图的最大匹配 47
5.9 欧拉回路/欧拉路径 47
5.10 连通性问题 48
5.10.1 边双连通分量/缩点 48
5.10.2 强连通分量/缩点 49
5.11 网络流 50
5.11.1 最大流 50
5.11.2 最小割 51
5.11.3 费用流 52
6 经典动态规划 53
6.1 优化方法 53
6.2 背包问题 53

6.2.1 01 背包 53
6.2.2 完全背包 53
6.2.3 二进制优化多重背包 54
6.2.4 分组背包 54
6.3 子序列问题 54
6.3.1 最长公共子序列 54
6.3.2 最长上升子序列 LIS 55
7 杂 55
7.1 库函数 55
7.2 取模类 55
7.3 读入一行数组（个数未知） 57
7.4 安全访问 mp 57
7.5 vector<PII> 赋值 vector<int> 57
7.6 正则括号序列（RBS） 57
7.6.1 RBS 建树 57
7.7 维护 mex 58
7.7.1 基础 mex 58
7.7.2 动态 mex 58
7.7.3 离线莫队 mex 58

本模板主要参考了 WIDA 算法模板，jiangly 算法模板

竞赛框架

基础框架

```cpp
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define CMAX(x, y) (x > y ? false : (x = y, true))
#define CMIN(x, y) (x < y ? false : (x = y, true))
#define ALL(x) (x).begin(), (x).end()

using LL = long long;
#define int LL
using i64 = long long;
using LD = long double;
template <typename T>
using PQ = priority_queue<T>;
template <typename T>
using PQ_ = priority_queue<T, vector<T>, greater<T>>;
using PII = pair<int, int>;

// pair 输出
template <typename T1, typename T2>
ostream &operator<<(ostream &os, const pair<T1, T2> &p)
{
    return os << "(" << p.first << ", " << p.second << ")";
}
// is_container 检测
template <typename T, typename = void>
struct _is_container : false_type
{
};
template <typename T>
struct _is_container<T, void_t<typename T::iterator, typename T::value_type, decltype(declval<T>().begin()),
     decltype(declval<T>().end())>> : true_type
{
};
template <typename T>
constexpr bool is_container_v = _is_container<T>::value;
// 通用容器输出
template <typename T, enable_if_t<is_container_v<T> && !is_same_v<T, string>, int> = 0>
```

```
ostream &operator<<(ostream &os, const T &v)
{
    os << "[";
    for (auto it = v.begin(); it != v.end(); ++it)
        os << (it != v.begin() ? " " : "") << *it;
    return os << "]";
}
// priority_queue 输出
template <typename T, typename Container, typename Compare>
ostream &operator<<(ostream &os, priority_queue<T, Container, Compare> pq)
{
    os << '[';
    while (!pq.empty())
    {
        os << pq.top() << (pq.size() == 1 ? "" : " ");
        pq.pop();
    }
    return os << ']';
}
// 变量/容器调试
#define debug(x) cerr << #x << "=" << (x) << endl;
// 二进制调试
#define bin8(x) cerr << #x << "=" << bitset<8>(x) << endl;
#define cerr \
    if (1)   \
    cerr // 调试输出开关

constexpr LL INF = 0x3f3f3f3f, MOD = 998244353;
inline void solve()
{
    /*
    */
}
signed main()
{
    ios_base::sync_with_stdio(false), cout.tie(nullptr), cin.tie(nullptr);
    int T = 1;
    cin >> T;
    while (T--)
    {
        solve();
    }
}
```

对拍器/交互器/数据生成器

对拍器

```
//linux
#include <bits/stdc++.h>
using namespace std;
int main() {
    for (int i = 1; i <= 10000; i++) {
        system("./gen > in");
        system("./sol < in > ans");
        system("./brute < in > out");
        if (system("diff -q ans out")) { printf("WA %d\n", i); return 0; }
        if (i % 100 == 0) printf("%d\n", i);
    }
    puts("AC");
}

//windows
#include <bits/stdc++.h>
using namespace std;
int main()
{
    for (int i = 1; i <= 10000; i++)
    {
        system("gen.exe > in");
        system("sol.exe < in > ans");
        system("brute.exe < in > out");
        if (system("fc ans out >nul")){ printf("WA %d\n", i); return 0; }
        if (i % 100 == 0) printf("%d\n", i);
    }
    puts("AC");
```



```
}
```

数据生成器

```
#include <bits/stdc++.h>
using namespace std;
#define int LL
using LL = long long;
mt19937 rng(time(0));
int rnd(int min = 0, int max = 1e9)
{ // 随机数生成器
    return uniform_int_distribution<int>(min, max)(rng);
}
// ---------- 常用数据形态 ----------

// 随机排列：1..n 的随机排列
inline vector<int> rand_perm(int n) {
    vector<int> p(n);
    iota(p.begin(), p.end(), 1);
    shuffle(p.begin(), p.end(), rng);
    return p;
}

// 随机树：n 个节点，返回 n-1 条边 (fa, i)，天然合法
// kind: 0 = 随机（父节点取 [1, i-1]），1 = 链，2 = 菊花
inline vector<pair<int, int>> rand_tree(int n, int kind = 0) {
    vector<pair<int, int>> e;
    e.reserve(max(0LL, n - 1));
    for (int i = 2; i <= n; i++) {
        int fa;
        if (kind == 1) fa = i - 1;   // 链
        else if (kind == 2) fa = 1;  // 菊花
        else fa = rnd(1, i - 1);     // 随机
        e.push_back({fa, i});
    }
    return e;
}

// 随机字符串：长度 n，字符从 charset 中取
inline string rand_string(int n, const string& charset = "abcdefghijklmnopqrstuvwxyz") {
    string s(n, ' ');
    for (int i = 0; i < n; i++) s[i] = charset[rnd(0, (int)charset.size() - 1)];
    return s;
}

// 随机简单无向图：n 点 m 边，无自环、无重边，返回边集
// 注意：m 不能超过 n*(n-1)/2；太密时可能提前退出，返回边数 < m 即放不下
inline vector<pair<int, int>> rand_graph(int n, int m, int max_attempts = 10000000) {
    vector<pair<int, int>> e;
    set<pair<int, int>> used;
    int tries = 0;
    while ((int)e.size() < m && ++tries <= max_attempts) {
        int u = rnd(1, n), v = rnd(1, n);
        if (u == v) continue;
        if (u > v) swap(u, v);
        if (used.insert({u, v}).second) e.push_back({u, v});
    }
    return e;
}

// 随机区间：返回 [l, r]，保证 1 <= l <= r <= n
inline pair<int, int> rand_interval(int n) {
    int l = rnd(1, n);
    return {l, rnd(l, n)};
}
int main(int argc, char *argv[])
{
    //多测
    int T = 1;
    cout << T << endl;
    while (T--)
    {

    }
    return 0;
```

```
}
```

交互器

- 交互题一般从二分查找入手，通过一些条件每次可以筛选掉一半的数据，将搜索空间折半，最终达到O(nlogn)的复杂度

```
static mt19937_64 rng(time(0));
int ra(int min = 0, int max = 1e9)
{ // 随机数生成器
    return std::uniform_int_distribution<int>(min, max)(rng);
}
struct interaction
{ // 交互器
//警示后人:
//记得关define endl '\n'
//记得开同步流
//先手动造数据，如果被卡了再造随机数据
    int cnt=0, MAXQUES=INT_MAX, submit;
    interaction(int is_sub):submit(is_sub) // 交互器，传参为1是提交模式，传参为0是交互测试模式
    {
        istream &input = is_sub ? cin : generator();
    }
   stringstream &generator() // 数据生成器，在这里生成数据
    {
        static stringstream ss(R"()");
        //**建议造固定数据调试成功后，再造随机数据验证**
        /*
        MAXQUES=INT_MAX;
        */
        return ss;
    }
    int ques(/*[询问传参]*/)// 询问
    {
        cout << "? " << endl; // 提问输出
        if (submit == 1)      // 提交模式，从stdin获得数据
        {
            int t;
            cin >> t;
            return t;
        }

        // 测试模式
        cnt++;
        if (cnt > MAXQUES)
        {
            cerr << "== Ques Cnt Limit ==" << endl;
            system("pause");
        }

        /*
        [交互逻辑]
        */
        cout << "reply>";
        /*
        [return reply];
        */
    }
    void ans(/*[答案传参]*/) // 回答
    {
        cout << "! " << endl; // 回答输出
        if (submit == 1)      // 提交模式: 回答后直接退出
            return;

        // 交互模式: 检查答案
        if (/*[答案错误]*/)//答案错误
        {
            cerr << "== Wrong Answer ==" << endl;
            cerr << "res=" << endl;
            cerr << "ques_cnt=" << cnt << endl;
            system("pause");
        }
        else
        {
            cerr << " > Accepted < \n";
```



```
        }
    }
};
//交互题要记得刷新输入输出缓冲区
```

数学

结论

组合数学

- **等差数列求和** $S_n = na_1 + n(n-1)d/2$
- **等比数列求和** $S_n = a_1(1-q^n)/(1-q)$
- **等比数列性质** $a_{i-1} * a_{i+1} = a_i^2$ 由于除法会丢精度，所以常用此方法检验等比数列

位运算

- **位加法** $a+b = a \oplus b + 2(a \ \& \ b) = a|b + a \ \& \ b = 2(a|b) - (a \oplus b)$
- **位减法** $a-b = (a \oplus b) - 2(!a \ \& \ b)$
- **位运算关系** $a \oplus b = (a|b) - (a \ \& \ b)$
- **判断一个数是不是 2 的幂次** $x \ \& \ (x-1) = 0$

重排不等式

反序和 $\leq$ 乱序和 $\leq$ 同序和

设两个递增序列：$a_1 \leq a_2 \leq ... \leq a_n, \quad b_1 \leq b_2 \leq ... \leq b_n$

对任意排列 $\sigma \in S_n$，有：

$\sum_{i=1}^{n} a_i b_{n-i+1} \ \leq \ \sum_{i=1}^{n} a_i b_{\sigma(i)} \ \leq \ \sum_{i=1}^{n} a_i b_i$

数论

1. 大于 2 的质数一定是奇数，而奇数一定是偶数 + 奇数（涉及到加减的质数性质）
2. 对于任何的 x 都有 $gcd(x, x+1) == 1$
3. 质数的幂一定互质
4. 1e9 以内的数最多有 9 个质因子

2.1.4.1 裴蜀定理

- 表述 1：$ax + by = k * gcd(a,b)$ a 和 b 的线性组合一定是 gcd 的倍数
- 表述 2：$ax + by = m$ 有解当且仅当 m 是 gcd(a,b) 的倍数

2.1.4.2 费马小定理

若 p 为质数且 $\gcd(a,p) = 1$，则 $a^{p-1} \equiv 1 \pmod{p}$

2.1.4.3 欧拉定理

若 $\gcd(a,m) = 1$，则 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 其中 $\varphi(m)$ 为小于 m 且与之互质的正整数个数

2.1.4.4 中国剩余定理（CRT）

求解同余方程组（m_i 两两互质）：

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_n \pmod{m_n} \end{cases}$$

令 $M=\prod m_i$，解为：

$$x \equiv \sum_{i=1}^{n} a_i \cdot \frac{M}{m_i} \cdot \left(\frac{M}{m_i}\right)^{-1} \pmod{M}$$

2.1.4.5 威尔逊定理 p 为质数 $\iff (p-1)! \equiv -1 \pmod{p}$

卡特兰数

是一类奇特的组合数，前几项为 $1,1,2,5,14,42,132,429,1430,4862$。如遇到以下问题，则直接套用即可。

- 【括号匹配问题】n 个左括号和 n 个右括号组成的合法括号序列的数量，为 Cat_n。
- 【进出栈问题】$1,2,\ldots,n$ 经过一个栈，形成的合法出栈序列的数量，为 Cat_n。
- 【二叉树生成问题】n 个节点构成的不同二叉树的数量，为 Cat_n。
- 【路径数量问题】在平面直角坐标系上，每一步只能**向上**或**向右**走，从 $(0,0)$ 走到 (n,n)，并且除两个端点外不接触直线 $y=x$ 的路线数量，为 $2Cat_{n-1}$。

计算公式：$Cat_n = \frac{C_{2n}^n}{n+1}$，$C_n = \frac{C_{n-1} * (4n-2)}{n+1}$。

随机数生成器

常用于异或哈希

```
//#include<random>
static std::mt19937_64 rng(std::random_device{}());
//或者  static std::mt19937_64 rng(time(0));
int rand() {
    int min=0,max=1e9;
    return std::uniform_int_distribution<int>(min, max)(rng);
}
```

位运算函数

返回二进制位数

```
//__lg()是gcc编译器，如果不是gcc编译器可以手写__lg()
int __lg(unsigned int x){//返回二进制最高位1的位置
    return 31 - __builtin_clz(x);
}
int bitlen(int x){
    return x==0?0:__lg(x)+1;
}
```

返回二进制的高位/低位

```
#define lowbit(x) (x&-x)
uLL higbit(uLL x)
{
    for (int i = 1; i <= 3<<sizeof(x); i <<= 1)
        x |= x >> i;
    return x - (x >> 1);
}
```

取整


```
//对a/b取整
a/b //向下取整
(a+b-1)/b //向上取整
(a+b/2)/b //四舍五入
```

快速幂 $O(logX)$

```
LL qpow(LL a, int b, int mod){
    LL res = 1 % mod;
    for (; b; b >>= 1, a = a * a % mod){
        if (b & 1) res = res * a % mod;
    }
    return res;
}
```

组合数学封装（组合 排列 阶乘 逆元 快速幂）

- 重复元素全排列 $\frac{n!}{n_1!n_2!\ldots n_k!}$
- 对称性：$C_n^k = C_n^{k-1}$
- 递推式：$C_n^k + C_n^{k-1} = C_{n+1}^k$
- 系数和：$\sum_{i=0}^{n} C_n^i = 2^n$
- 一般隔板（n 元素分 k 组）：$\boxed{\binom{n-1}{k-1}}$
- 空组隔板（允许隔板之间无元素）$\boxed{\binom{n+k-1}{k-1}}$

```
//受限于内存和运行时间，最大可以处理1e8的数据（不会出现爆int的问题）
class Comb // 组合数学套餐 组合 排列 阶乘 逆元 快速幂
{
    vector<int> f;
    vector<int> g;
    static constexpr int mod = 998244353;
public:
    LL qpow(LL a, int b){
        LL res = 1 % mod;
        for (; b; b >>= 1, a = a * a % mod)
            if (b & 1) res = res * a % mod;
        return res;
    }
    Comb(int n) : f(n + 1), g(n + 1)
    {
        f[0] = 1;
        for (int i = 1; i <= n; i++)
            f[i] = 1LL * f[i - 1] * i % mod;
        g[n] = qpow(f[n], mod - 2);
        for (int i = n; i; i--)
            g[i - 1] = 1LL * g[i] * i % mod;
    }
    int C(int n, int m)
    {
        if (n < m || m < 0)
            return 0;
        return 1LL * f[n] * g[m] % mod * g[n - m] % mod;
    }
    int A(int n, int m)
    {
        if (n < m || m < 0)
            return 0;
        return f[n] * g[n - m] % mod;
    }
    int fac(int x)
    {
        if (x < 0 || x > f.size())
            return -1;
        return f[x];
```

```
    }
    int inv(int x) { return qpow(x, mod - 2); }
};
Comb cmb(1e6);
```

质数判定

```
bool isprime(int n){//O (sqrt(N))
    if(n<2)return false;
    for(int i=2;i<=n/i;i++){
        if(n%i==0)return false;
    }
    return true;
}
```

快速预处理质因子 $O(NloglogN)$

```
vector<vector<int>> pfac(N + 1);
for (int i = 2; i <= N;i++){//从小到大计算每个数的质因子
    if(!pfac[i].empty())
        continue;
    for (int j = i; j <= N;j+=i)
        pfac[j].push_back(i);
}
```

欧拉筛

朴素欧拉筛

```
class sieve{
    vector<bool>st;
    vector<int> primes;
    public:
    sieve(int n){
        st.assign(n + 1, false);
        for (int i = 2;i<=n;i++){
            if(!st[i])primes.push_back(i);
            for (int j = 0; primes[j] <= n / i;j++){
                st[primes[j] * i] = true;
                if(i%primes[j]==0)
                    break;
                }
        }
    }
    int find(int x){//找到大于等于x的第一个质数
        return *lower_bound(primes.begin(), primes.end(),x);
    }
};
sieve sv(1e6);
```

SPF 欧拉筛（支持快速质因数分解 O(logn)，快速质数判断 O(1)）

```
struct sieve
{
    vector<int> spf;//每个数的最小质因子
    vector<int> primes;
    int n;
    sieve(int _n) : n(_n)
    {
        spf.resize(n + 1, 0);
        spf[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (spf[i] == 0)
            {
```



```
                spf[i] = i; // 质数的最小质因子是它本身
                primes.push_back(i);
            }
            for (int p : primes)
            {
                if (p > spf[i] || 1LL * i * p > n)
                    break;
                spf[i * p] = p; // 标记最小质因子
            }
        }
    }
    // 质因数分解O(logn)
    set<int> divide(int n)
    {
        set<int> res;
        while (n > 1)
        {
            int p = spf[n], count = 0;
            while (spf[n] == p)
                count++, n /= p;
            res.insert(p);
            // 在这里从小到大分解出了每一个质因数p^count
        }
        return res;
    }
    bool isprime(int x)
    {
        return spf[x] == x;
    }
    int find(int x)
    { // 找到大于等于x的第一个质数
        return *lower_bound(primes.begin(), primes.end(), x);
    }
};
sieve sv(1e6);
```

最大公约数 最小公倍数

- **gcd 的线性组合不变量**：对于任何整数集合，它们的 gcd 等于这些整数的所有线性组合的 gcd。也就是说，如果一组数可以通过另一组数的线性组合表示，那么这两组数的 gcd 相同。说人话就是：**gcd 内部的数可以随意互相加减**

```
int gcd(int a,int b){//返回a和b的最大公约数 O(logn)
    return b ? gcd(b, a % b) : a;
}
int gcd(int a, int b)//异或优化快速gcd
{
    if(b) while(b ^= a ^= b ^= a %= b);
    return a;
}
inline LL gcd(LL a, LL b) // 移位优化最速gcd（速度跟C++17内置gcd相当）
{
#define tz __builtin_ctzll
    a = a < 0 ? -a : a, b = b < 0 ? -b : b;
    if (!a || !b)
        return a | b;
    int t = tz(a | b);
    a >>= tz(a);
    while (b){
        b >>= tz(b);
        if (a > b)
            b^=a^=b^=a;
        b -= a;
    }
    return a << t;
#undef tz;
}
int lcm(int a, int b){
    return a / gcd(a, b)*b;//警示后人:先除再乘防止越界
}
```

拓展欧几里得算法

```
int exgcd(int a,int b,int &x,int &y){
    if(!b){
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x);
    y -= a / b % x;
    return d;
}
```

乘法逆元

费马小定理解（快速幂）

```
LL inv (LL x){ return qpow(x,mod-2,mod);}
```

拓展欧几里得解 $O(logX)$

mod 没有限制，比快速幂常数大一点

```
int exgcd(int a,int b,int &x,int &y){
    if(!b){
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
LL inv(int a,int mod){
    int x,y,g = exgcd(a, mod, x, y);
    return g==1 ? (x % mod + mod) % mod:-1;
}
```

（离线）线性递推解 $O(N)$

```
inv[1] = 1;
for (int i = 2; i <= n; i++ )
    inv[i] = (p - p / i) * inv[p % i] % p;
```

异或线性基

```
struct LinearBasis
{
    int MAXL;
    vector<LL> a;
    LinearBasis()
    {
        MAXL = 64;
        a.assign(MAXL+1, 0);
    }
    LinearBasis(vector<LL> &arr, int _MAXL = 64) : a(_MAXL+1), MAXL(_MAXL)
    {
        build(arr);
    }
    void insert(LL t)
    {
        for (int j = MAXL; j >= 0; j--)
        {
            if (!t)
                return;
            if (!(t & (1LL << j)))
```



```
                continue;
            if (a[j])
                t ^= a[j];
            else
            {
                for (int k = 0; k < j; k++)
                    if (t & (1LL << k)) // 消除t中的低位中已占的最高位
                        t ^= a[k];
                for (int k = j + 1; k <= MAXL; k++)//消除高位中t的最高位
                    if (a[k] & (1LL << j))
                        a[k] ^= t;
                a[j] = t;
                break;
            }
        }
    }
    void build(vector<LL>& x){
        a.assign(MAXL, 0);
        for(int i=1;i<x.size();i++)
            insert(x[i]);
    }
    LL queryMax(){
        LL res=0;
        for(int i=0;i<=MAXL;i++)res^=a[i];
        return res;
    }
    void merge(const LinearBasis &other){
        for(int i=0;i<=MAXL;i++)
            insert(other.a[i]);
    }
};
```

矩阵

矩阵封装

```
inline int qpow(LL n, LL k, int p = MOD)
{
    LL r = 1;
    for (; k; k >>= 1, n = n * n % p)
        if (k & 1)
            r = r * n % p;
    return r;
}
/* ==================== 矩阵板子 (下标 0..n-1, 阶数运行时定) ====================
 * Matrix A;                    // 空矩阵
 * Matrix A(n);                 // n*n 零矩阵
 * Matrix A(n, v);              // n*n 全都是 v
 * A.clear();  A.reset();       // 清零 / 置为单位阵
 * A * B     A + B              // 乘法 / 加法
 * A.pow(k)                     // 矩阵快速幂, k 到 1e18 都行
 * A.getinv()                   // 求 A 的逆, 不可逆返回零矩阵且 A.ok = 0
 * A.show()                     // 打到 cerr, 调试用
 * ========================================================================== */
struct Matrix
{
    int n;
    vector<vector<LL>> M;
    Matrix(int n = 0, LL v = 0) : n(n), M(n, vector<LL>(n, v)) {}
    void clear()
    {
        for (auto &r : M)
            fill(r.begin(), r.end(), 0LL);
    }
    void reset()
    { // 初始化
        clear();
        for (int i = 0; i < n; ++i)
            M[i][i] = 1;
    }
    Matrix friend operator*(const Matrix &A, const Matrix &B)
    {
        assert(A.n == B.n);
        Matrix Ans(A.n);
        for (int i = 0; i < A.n; ++i)
```

```
        {
            auto &Ri = Ans.M[i];
            for (int k = 0; k < A.n; ++k)
            {
                LL a = A.M[i][k];
                if (!a)
                    continue;
                const auto &Bk = B.M[k];
                for (int j = 0; j < A.n; ++j)
                    Ri[j] = (Ri[j] + a * Bk[j]) % MOD;
            }
        }
        return Ans;
    }
    Matrix friend operator+(const Matrix &A, const Matrix &B)
    {
        assert(A.n == B.n);
        Matrix Ans(A.n);
        for (int i = 0; i < A.n; ++i)
            for (int j = 0; j < A.n; ++j)
                Ans.M[i][j] = (A.M[i][j] + B.M[i][j]) % MOD;
        return Ans;
    }
    Matrix pow(LL k) const
    { // 矩阵快速幂
        Matrix Ans(n), b = *this;
        Ans.reset();
        for (; k; k >>= 1, b = b * b)
            if (k & 1)
                Ans = Ans * b;
        return Ans;
    }
    bool ok = 1;
    Matrix getinv()
    { // 矩阵求逆
        Matrix res(n);
        vector<vector<LL>> a(n, vector<LL>(2 * n, 0)); // 增广矩阵 [A|I]，列要开到 2*n
        ok = 1;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = M[i][j];
        for (int i = 0; i < n; i++)
            a[i][i + n] = 1;
        for (int i = 0; i < n; i++)
        {
            int pos = i;
            while (pos < n && !a[pos][i])
                pos++; // 找一个非零主元
            if (pos == n)
            {
                ok = 0;
                return res; // 这一列全 0，不可逆
            }
            swap(a[i], a[pos]);
            LL inv = qpow(a[i][i], MOD - 2);
            for (int j = 0; j < n; j++)
                if (j != i && a[j][i])
                {
                    LL mul = a[j][i] * inv % MOD;
                    for (int k = i; k < 2 * n; k++)
                        a[j][k] = ((a[j][k] - a[i][k] * mul) % MOD + MOD) % MOD;
                }
            for (int j = 0; j < 2 * n; j++)
                a[i][j] = a[i][j] * inv % MOD;
        }
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                res.M[i][j] = a[i][n + j];
        return res;
    }
    void show()
    {
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                cerr << M[i][j] << "\t\n"[j == n - 1];
    }
};
```


基础算法

最优化方法

整数域

3.1.1.1 二分

3.1.1.1.1 左查找

```
auto bsearch = [&](LL l, LL r, const function<bool(LL)> &check){
    while (l < r){
        LL mid = l + (r - l >> 1);
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return check(l) ? l : -1;
};
```

3.1.1.1.2 右查找

```
auto bsearch = [&](LL l, LL r, const function<bool(LL)> &check){
    while (l < r){
        LL mid = l + (r - l + 1 >> 1);
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return check(l) ? l : -1;
};
```

3.1.1.1.3 单峰函数二分

```
auto func = [&](int x) -> int
{
    return 0;
};
auto bsearch = [&](LL L, LL R)
{
    while (L < R)
    {
        int MID = L + R >> 1;
        if (func(MID) > func(MID + 1))
            R = MID;
        else
            L = MID + 1;
    }
    return L;
};
```

实数域

3.1.2.1 浮点数二分

```
const double eps = 1e-6;   // eps 表示精度，取决于题目对精度的要求
while (r - l > eps){
    double mid = (l + r) / 2;
    if (check(mid)) r = mid;
    else l = mid;
}
return l;
```

另一种写法（限制二分次数）

```
for (int t = 1; t <= 100; t++){
    long double mid = (l + r) / 2;
    if (check(mid))
```

```
        r = mid;
    else
        l = mid;
}
```

3.1.2.2 实数域三分 限制次数实现

```
long double l=-1e9,r=1e9;
for(int t = 1; t <= 100; t++){
    long double mid1 = (l*2 + r) / 3;
    long double mid2 = (l + r*2) / 3;
    if(check(mid1)<check(mid2))r=mid2;
    else l=mid1;
}
```

前缀和与差分

- 一维前缀和

```
s[i]=a[i]+s[i-1];//前缀和
s[r]-s[l-1]//[l,r]的区间和
```

- 一维差分

```
b[i]=a[i]-a[i-1];//若a为前缀和数组，差分后则为原数组
b[l]+=c,b[r+1]-=c;//在区间[l,r]加c
```

- 二维前缀和

```
//构建前缀和数组
for (int i=1; i<=n; i++)
    for (int j=1; j<=m; j++)
        s[i][j] = s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j];
//求[x1,y1],[x2,y2]子矩阵和
sum=s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1];
```

- 二维差分

```
void insert(int x1,int y1,int x2,int y2,int c)
{     //对b数组执行插入操作，等价于对a数组中的(x1,y1)到(x2,y2)之间的元素都加上了c
    b[x1][y1] += c;
    b[x2 + 1][y1] -= c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y2 + 1] += c;
}
//构建差分数组
for(int i = 1;i <= n;i++)
    for(int j = 1;j <= m;j++)
        insert(i, j, i, j, a[i][j]);
//或者
b[i][j] = a[i][j] - a[i - 1][j] - a[i][j - 1] + a[i - 1][j - 1]
```

高精度

```
struct BigInt {
    const int N=1000;
    int a[N];
    BigInt(int x = 0) : a{} {
        for (int i = 0; x; i++) {
            a[i] = x % 10;
            x /= 10;
        }
    }
```



```
    BigInt &operator*=(int x) {
        for (int i = 0; i < N; i++) {
            a[i] *= x;
        }
        for (int i = 0; i < N - 1; i++) {
            a[i + 1] += a[i] / 10;
            a[i] %= 10;
        }
        return *this;
    }
    BigInt &operator/=(int x) {
        for (int i = N - 1; i >= 0; i--) {
            if (i) {
                a[i - 1] += a[i] % x * 10;
            }
            a[i] /= x;
        }
        return *this;
    }
    BigInt &operator+=(const BigInt &x) {
        for (int i = 0; i < N; i++) {
            a[i] += x.a[i];
            if (a[i] >= 10) {
                a[i + 1] += 1;
                a[i] -= 10;
            }
        }
        return *this;
    }
};
std::ostream &operator<<(std::ostream &o, const BigInt &a) {
    int t = N - 1;
    while (a.a[t] == 0) {
        t--;
    }
    for (int i = t; i >= 0; i--) {
        o << a.a[i];
    }
    return o;
}
```

离散化

版本 1

```
vector<int> alls; // 存储所有待离散化的值
sort(alls.begin(), alls.end()); // 将所有值排序
alls.erase(unique(alls.begin(), alls.end()), alls.end());   // 去掉重复元素

// 二分求出x对应的离散化的值
int idx(int x){ // 找到第一个大于等于x的位置
    int l = 0, r = alls.size() - 1;
    while (l < r)
    {
        int mid = l + r >> 1;
        if (alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1; // 映射到1, 2, ...n
}
//或者
lower_bound(alls.begin(),alls.end(),x);
//或者用map映射储存映射值
```

版本 2

```
//数组版离散化
map<int,int>idx;//用来储存映射值
sort(b+1,b+1+btop);//排序
b[0] = -0x3f3f3f3f;//b数组用来暂存待离散化的值，离散化后b就没用了
for (int i = 1; i <= btop;i++){//去重
    if(b[i]!=b[i-1])
        alls[++top] = b[i], idx[b[i]] = top;
```

```
}
```

版本 3

```
//另一种离散化
function<void(vector<int> &)> work = [&](vector<int> &vec) -> void
{
    vector<int> p(vec.size());
    iota(p.begin(), p.end(), 0);
    sort(p.begin() + 1, p.end(), [&](int x, int y)
         { return vec[x] < vec[y]; });
    for (int i = 1; i <vec.size();i++)
        vec[p[i]] = i;
};
```

排序

归并排序

```
void merge_sort(int q[], int l, int r)
{
    if(l>=r)return;
    int mid = r + l >> 1;
    merge_sort(q, l, mid), merge_sort(q, mid + 1, r);//左右两边都有序了
    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r)
        if (q[i] <= q[j])tmp[k++] = q[i++];
        else tmp[k++] = q[j++];
    while (i <= mid)tmp[k++] = q[i++];
    while (j <= r)tmp[k++] = q[j++];
    for (int i = l, j = 0; i <= r; i++, j++)q[i] = tmp[j];
}
```

快速排序

```
void Qsort(int a[], int l, int r){
    int i = l, j = r, key = a[l + r>>1];
    do {
        while (a[i] < key)i++;
        while (a[j] > key)j--;
        if (i <= j)swap(&a[i++], &a[j--]);
    } while (i <= j);
    if (l < j)Qsort(a, l, j);
    if (i < r)Qsort(a, i, r);
}
```

快速读入

```
#define rd read()
inline long long read()
{
    long long x=0,y=1;char c=getchar();//y代表正负（1.-1），最后乘上x就可以了。
    while (c<'0'||c>'9') {if (c=='-') y=-1;c=getchar();}//如果c是负号就把y赋为-1
    while (c>='0'&&c<='9') x=x*10+c-'0',c=getchar();
    return x*y;//乘起来输出
}
```

回滚莫队

以”维护 区间 超过阈值频率最高的元素“为例 3636. 查询超过阈值频率最高元素

```
class Solution {
public:
```



```
    vector<int> subarrayMajority(vector<int>& nums, vector<vector<int>>& queries) {
        int n = nums.size(), m = queries.size();

        unordered_map<int, int> cnt;
        int max_cnt = 0, min_val = 0;

        // 添加元素 x
        auto add = [&](int x) {
            int c = ++cnt[x];
            if (c > max_cnt) {
                max_cnt = c;
                min_val = x;
            } else if (c == max_cnt) {
                min_val = min(min_val, x);
            }
        };

        vector<int> ans(m, -1);
        int block_size = ceil(n / sqrt(m * 2));

        struct Query {
            int bid, l, r, threshold, qid; // [l,r) 左闭右开
        };

        vector<Query> qs;
        for (int i = 0; i < m; i++) {
            auto& q = queries[i];
            int l = q[0], r = q[1] + 1, threshold = q[2]; // 左闭右开

            // 大区间离线（保证 l 和 r 不在同一个块中）
            if (r - l > block_size) {
                qs.emplace_back(l / block_size, l, r, threshold, i);
                continue;
            }

            // 小区间暴力
            for (int j = l; j < r; j++) {
                add(nums[j]);
            }
            if (max_cnt >= threshold) {
                ans[i] = min_val;
            }

            // 重置数据
            cnt.clear();
            max_cnt = 0;
        }

        ranges::sort(qs, {}, [](auto& q) { return pair(q.bid, q.r); });

        int r;
        for (int i = 0; i < qs.size(); i++) {
            auto& q = qs[i];
            int l0 = (q.bid + 1) * block_size;
            if (i == 0 || q.bid > qs[i - 1].bid) { // 遍历到一个新的块
                r = l0; // 右端点移动的起点
                // 重置数据
                cnt.clear();
                max_cnt = 0;
            }

            // 右端点从 r 移动到 q.r（q.r 不计入）
            for (; r < q.r; r++) {
                add(nums[r]);
            }

            int tmp_max_cnt = max_cnt, tmp_min_val = min_val;

            // 左端点从 l0 移动到 q.l（l0 不计入）
            for (int j = q.l; j < l0; j++) {
                add(nums[j]);
            }
            if (max_cnt >= q.threshold) {
                ans[q.qid] = min_val;
            }

            // 回滚
            max_cnt = tmp_max_cnt;
```

```
            min_val = tmp_min_val;
            for (int j = q.l; j < l0; j++) {
                cnt[nums[j]]--;
            }
        }
        return ans;
    }
};
```

数据结构

单调队列

```
//双端队列
deque<int>q
for(int i=0;i<n;i++){//队列中存的是下标
    if(q.size()&&i-k+1>q.front())q.pop_front();//判断队头是否已经滑出窗口;
    while(q.size()&&a[q.back()]>=a[i])tt--;//维护队列单调性 如果队尾大于要入列的元素，弹出队尾//双端队列
    q.push_back(i);新元素入队
}
```

单调栈

```
//找到每一个元素在入栈方向上第一个满足对该元素的单调条件的元素
stack<int> stk;
for(int i=0;i<n;i++){
    while(stk.size()&&a[stk.back()]>=a[i])stk.pop()//出栈 第一个小于等于a[stk.back()]的数是a[i]
    stk.push(i);//入栈//存的是下标
}
```

树状数组 Fenwick BIT

- BIT 只支持有逆运算的操作
- **如果只维护前缀，不维护区间，则不需要运算的可逆性**

半动态区间和（单点修改）

```
template <typename T>
struct BIT
{
    vector<T> c;
    int n;
    BIT(int n) : n(n), c(n + 1, T{}) {} // 树状数组大小为[1,n]
    BIT(vector<T> &a) :n(a.size()-1),c(n+1,0)// 用数组初始化
    {
        vector<T> pre(n + 1, 0);
        for (int i = 1; i <= n; i++)
        {
            pre[i] = pre[i - 1] + a[i];
            c[i] = pre[i] - pre[i - (i & -i)];
        }
    }
    T sum(int x)
    { // 求x位置的前缀和
        T res{};
        for (int i = x; i; i -= i & -i)
            res += c[i];
        return res;
    }
    void add(int x, const T &y)
    { // 在x位置上加上y
        for (int i = x; i <= n; i += i & -i)
            c[i] += y;
    }
    T range(int l, int r)
    { // 计算[l,r]的区间和
        return sum(r) - sum(l - 1);
    }
    int select(const T &k) // 找到最大的下标x,使得sum(x)<=k,维护频次信息时，select(k)+1表示第k大的数，复杂度O(logn) 比在sum上二分O(log^2 n)更快,比对顶堆常数更小
    {
        int x = 0;
        T cur{};
        for (int i = 1 << __lg(n); i; i /= 2)
            if (x + i <= n && cur + c[x + i] <= k)
                x += i, cur += c[x];
        return x;
    }
};
```


动态区间和（区间修改）

```
template <typename T>
struct BIT
{
    vector<T> a, b;
    int n;
    BIT(int n) : n(n), a(n + 1, T{}), b(n + 1, T{}) {} // 树状数组大小为[1,n]
    void apply(int l, int r, T val)
    { // 区间修改
        auto add = [&](int pos, T val)
        {
            for (int i = pos; i <= n; i += i & -i)
            {
                a[i] += val;
                b[i] += pos * val;
            }
        };
        add(l, val), add(r + 1, -val);
    }
    i64 query(int l, int r)
    { // 区间和查询
        auto sum = [&](int x)
        {
            T ans = {};
            for (int i = x; i; i -= i & -i)
                ans += (x + 1) * a[i] - b[i];
            return ans;
        };
        return sum(r) - sum(l - 1);
    }
};
```

ST 表（Sparse Table）

可修起始下标 要求 RMQ 的操作函数具有 == 区间可加性 ==，如 min，max，gcd，位操作（&，^，|），修改操作函数的时候记得修改 ans 的初始值 初始化时间复杂度为`O(n*logn)`，查询的复杂度为`O(1)`

```
class ST
{
    vector<vector<int>> f;
    int BG = 0; // 起始下标
    int n;

public:
    ST(const vector<int> &a)
    {
        n = a.size() - BG;
        f.assign(n + 1, vector<int>(int(log2(n)) + 2));
        for (int i = 1; i <= n; i++)
            f[i][0] = a[i - 1 + BG];
        for (int j = 1; (1 << j) <= n; j++)
            for (int i = 1; i + (1 << j) - 1 <= n; i++)
                f[i][j] = max(f[i][j - 1], f[i + (1 << j - 1)][j - 1]); // op
    }
```

```
    int RMQ(int l, int r)
    {
        if (l > r) // 非法情况
            return 0;
        l = clamp(l, BG, n), r = clamp(r, BG, n);
        l = l - BG + 1, r = r - BG + 1;
        int L = r - l + 1, ans = -0x3f3f3f3f; // op
        for (int i = 0, j = l; (1 << i) <= L; i++)
        {
            if ((L >> i) & 1)
            {
                ans = max(ans, f[j][i]); // op
                j += (1 << i);
            }
        }
        return ans;
    }
};
```

线段树

ACL 线段树（好用，推荐，全能）

```
/* ==================== 懒标记线段树板子（非递归版） ====================
 * 要求: C++20 (用到了 <bit> 的 bit_ceil / countr_zero)
 * 模板参数 (函数以指针传入):
 *   S: 元素类型 | op: S(S,S) | e: S()
 *   F: 懒标记类型 | mapp: S(F,S) | cpt: F(F,F) | id: F()
 * 接口 (n 为序列长度, 区间均半开 [l, r)):
 *   构造: lseg(n) / lseg(vector<S>)   O(n)
 *   set(p, x)        单点赋值                    O(log n)
 *   get(p)           单点查询                    O(log n)
 *   prod(l, r)       区间聚合                    O(log n)
 *   all_prod()       全区间聚合                  O(1)
 *   apply(p, f)      单点打懒标记                O(log n)
 *   apply(l, r, f)   区间打懒标记                O(log n)
 *   max_right(l, g)  最大的 r 使 [l, r) 满足 g   O(log n)  需 g(e())
 *   min_left(r, g)   最小的 l 使 [l, r) 满足 g   O(log n)  需 g(e())
 * 越界调用为未定义行为, 主程序里自己保证。
 * ================================================================== */

template <class S, auto op, auto e, class F, auto mapp, auto cpt, auto id>
struct lseg
{
public:
    lseg() : lseg(0) {}
    lseg(int n) : lseg(vector<S>(n, e())) {}
    lseg(const vector<S> &v) : _n(int(v.size()))
    {
        size = (int)bit_ceil((unsigned)_n);
        log = countr_zero((unsigned)size);
        d = vector<S>(2 * size, e());
        lz = vector<F>(size, id());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
        {
            update(i);
        }
    }

    void set(int p, S x) // 0 <= p < n
    {
        p += size;
        for (int i = log; i >= 1; i--)
            push(p >> i);
        d[p] = x;
        for (int i = 1; i <= log; i++)
            update(p >> i);
    }

    S get(int p) // 0 <= p < n
    {
        p += size;
```



```
        for (int i = log; i >= 1; i--)
            push(p >> i);
        return d[p];
    }

    S prod(int l, int r) // 0 <= l <= r <= n
    {
        if (l == r)
            return e();

        l += size;
        r += size;

        for (int i = log; i >= 1; i--)
        {
            if (((l >> i) << i) != l)
                push(l >> i);
            if (((r >> i) << i) != r)
                push((r - 1) >> i);
        }

        S sml = e(), smr = e();
        while (l < r)
        {
            if (l & 1)
                sml = op(sml, d[l++]);
            if (r & 1)
                smr = op(d[--r], smr);
            l >>= 1;
            r >>= 1;
        }

        return op(sml, smr);
    }

    S all_prod() { return d[1]; }

    void apply(int p, F f) // 0 <= p < n
    {
        p += size;
        for (int i = log; i >= 1; i--)
            push(p >> i);
        d[p] = mapp(f, d[p]);
        for (int i = 1; i <= log; i++)
            update(p >> i);
    }
    void apply(int l, int r, F f) // 0 <= l <= r <= n
    {
        if (l == r)
            return;

        l += size;
        r += size;

        for (int i = log; i >= 1; i--)
        {
            if (((l >> i) << i) != l)
                push(l >> i);
            if (((r >> i) << i) != r)
                push((r - 1) >> i);
        }

        {
            int l2 = l, r2 = r;
            while (l < r)
            {
                if (l & 1)
                    all_apply(l++, f);
                if (r & 1)
                    all_apply(--r, f);
                l >>= 1;
                r >>= 1;
            }
            l = l2;
            r = r2;
        }

        for (int i = 1; i <= log; i++)
        {
            if (((l >> i) << i) != l)
```

```
            update(l >> i);
        if (((r >> i) << i) != r)
            update((r - 1) >> i);
    }
}

template <bool (*g)(S)>
int max_right(int l)
{
    return max_right(l, [](S x)
                     { return g(x); });
}
template <class G>
int max_right(int l, G g) // 0 <= l <= n, 需 g(e()) == true
{
    if (l == _n)
        return _n;
    l += size;
    for (int i = log; i >= 1; i--)
        push(l >> i);
    S sm = e();
    do
    {
        while (l % 2 == 0)
            l >>= 1;
        if (!g(op(sm, d[l])))
        {
            while (l < size)
            {
                push(l);
                l *= 2;
                if (g(op(sm, d[l])))
                {
                    sm = op(sm, d[l]);
                    l++;
                }
            }
            return l - size;
        }
        sm = op(sm, d[l]);
        l++;
    } while ((l & -l) != l);
    return _n;
}

template <bool (*g)(S)>
int min_left(int r)
{
    return min_left(r, [](S x)
                    { return g(x); });
}
template <class G>
int min_left(int r, G g) // 0 <= r <= n, 需 g(e()) == true
{
    if (r == 0)
        return 0;
    r += size;
    for (int i = log; i >= 1; i--)
        push((r - 1) >> i);
    S sm = e();
    do
    {
        r--;
        while (r > 1 && (r % 2))
            r >>= 1;
        if (!g(op(d[r], sm)))
        {
            while (r < size)
            {
                push(r);
                r = 2 * r + 1;
                if (g(op(d[r], sm)))
                {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
```



```
        } while ((r & -r) != r);
        return 0;
    }

private:
    int _n, size, log;
    vector<S> d;
    vector<F> lz;

    void update(int k) { d[k] = op(d[2 * k], d[2 * k + 1]); }
    void all_apply(int k, F f)
    {
        d[k] = mapp(f, d[k]);
        if (k < size)
            lz[k] = cpt(f, lz[k]);
    }
    void push(int k)
    {
        all_apply(2 * k, lz[k]);
        all_apply(2 * k + 1, lz[k]);
        lz[k] = id();
    }
};

/* ==================== 现成实现: 区间加 / 区间推平 + 最小值 (兼区间和) ====================
 * 示例:
 *   vector<S> a(n);              // 每个元素 {值, 值, 1}: sum/mn 都是该值, len=1 (len 不能漏)
 *   for (int i = 0; i < n; i++) a[i] = {x[i], x[i], 1};
 *   SegT seg(a);                 // 建树 O(n)
 *   seg.apply(l, r, F{1, d});    // 区间 [l, r) 整体 +d
 *   seg.apply(l, r, F{2, c});    // 区间 [l, r) 推平为 c
 *   seg.prod(l, r).mn;           // 查区间最小值
 *   seg.prod(l, r).sum;          // 查区间和
 *   seg.set(p, {y, y, 1});       // 单点 p 改成 y
 *   seg.get(p).mn;               // 单点查
 * ========================================================== */
// 聚合元素 — 节点要维护什么信息 (区间和 / 区间最小值 / 区间长度)
struct S
{
    long long sum, mn;
    int len;
};
// op: 聚合方式 — 父节点 = 左儿子 op 右儿子 (两个儿子的信息怎么合并)
S op(S a, S b) { return {a.sum + b.sum, min(a.mn, b.mn), a.len + b.len}; }
// e: 单位元 — 空区间的聚合值, 建树填充和 prod 累加初值都用它 (mn 取 INF, len 取 0)
S e() { return {0, (long long)4e18, 0}; }

// F: 懒标记 — kind 是操作类型 (0 无操作 | 1 区间加 | 2 推平), v 是操作值
struct F
{
    int kind;
    long long v;
};

// mapp: 打标记 — 懒标记 f 作用于元素 x, 返回这个节点打标后聚合的新值
S mapp(F f, S x)
{
    if (f.kind == 0)
        return x; // 无操作, 原样返回
    if (f.kind == 1)
        return {x.sum + f.v * x.len, x.mn + f.v, x.len}; // 区间加: 和 +v*len, 最小值 +v
    return {f.v * x.len, f.v, x.len};                    // 推平: 整段都变成 v
}
// cpt: 懒标记碰撞 — 节点已有旧标记 g, 又打上新标记 f, 合并成一个新标记 (后打的优先级高)
F cpt(F f, F g)
{
    if (f.kind == 2)
        return f; // 新的推平盖掉旧标记, 旧标记无论是什么都失效
    if (f.kind == 1)
    {
        if (g.kind == 2)
            return F{2, g.v + f.v}; // 先推平 c 再统一加 d ⇒ 等价于推平 c+d
        if (g.kind == 1)
            return F{1, g.v + f.v}; // 加 + 加 ⇒ 加, 两个值相加
        return f;                   // 旧标记为空, 只留新加的
    }
    return g; // 新标记为空, 保留旧标记
```

```
}
// id: 空懒标记 — 无操作 (kind=0), 建树时所有懒标记的初始值
F id() { return {0, 0}; }

using SegT = lseg<S, op, e, F, mapp, cpt, id>;
```

泛型线段树（自定义 Info，Merge，单点修改，区间查询，无 lazy）

```
template <typename Info, typename MergeOp>
struct SegTree
{
    int n;
    vector<Info> seg;
    [[no_unique_address]] MergeOp merge; // 空类不占空间

    SegTree(int n_, MergeOp merge_ = {}) : n(n_), merge(move(merge_))
    {
        seg.assign(4 << __lg(n), Info());
        build(1, 1, n);
    }
    SegTree(const vector<Info> &a, MergeOp merge_ = {}) : n(a.size() - 1), merge(move(merge_))
    {
        seg.assign(4 << __lg(n), Info());
        build(1, 1, n, a);
    }

    void pull(int p) { seg[p] = merge(seg[p * 2], seg[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<Info> *a = nullptr)
    {
        if (l == r)
        {
            if (a)
                seg[p] = (*a)[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }

    void upd(int p, int l, int r, int pos, const Info &val)
    {
        if (l == r)
        {
            seg[p] = val;
            return;
        }
        int m = (l + r) / 2;
        if (pos <= m)
            upd(p * 2, l, m, pos, val);
        else
            upd(p * 2 + 1, m + 1, r, pos, val);
        pull(p);
    }
    void upd(int pos, const Info &val) { upd(1, 1, n, pos, val); }

    Info qry(int p, int l, int r, int ql, int qr)
    {
        if (ql <= l && r <= qr)
            return seg[p];
        int m = (l + r) / 2;
        if (qr <= m)
            return qry(p * 2, l, m, ql, qr);
        if (ql > m)
            return qry(p * 2 + 1, m + 1, r, ql, qr);
        return merge(qry(p * 2, l, m, ql, qr), qry(p * 2 + 1, m + 1, r,
                                                   ql, qr));
    }
    Info qry(int l, int r) { return qry(1, 1, n, l, r); }
};
```

需要在 solve 里实现：


```
struct Info
{

};
auto merge = [&](const Info &a, const Info &b)
{

};
SegTree<Info, decltype(merge)> seg(n, move(merge));
```

WIDA 模板线段树

```
template<class T> struct Segt {
    struct node {
        int l, r;
        T w, rmq, lazy;
        bool tg;                    // 是否有未下传的懒标记（加法/赋值通用）
    };
    vector<T> w;
    vector<node> t;

    Segt() {}
    Segt(int n) { init(n); }
    Segt(vector<int> in) {
        int n = in.size() - 1;
        w.resize(n + 1);
        for (int i = 1; i <= n; i++) {
            w[i] = in[i];
        }
        init(in.size() - 1);
    }

    #define GL (k << 1)
    #define GR (k << 1 | 1)

    void init(int n) {
        w.resize(n + 1);
        t.resize(n * 4 + 1);
        auto build = [&](auto self, int l, int r, int k = 1) {
            if (l == r) {
                t[k] = {l, r, w[l], w[l], 0, false};
                return;
            }
            t[k] = {l, r, 0, 0, 0, false};
            int mid = (l + r) / 2;
            self(self, l, mid, GL);
            self(self, mid + 1, r, GR);
            pushup(k);
        };
        build(build, 1, n);
    }
    void pushdown(node &p, T lazy) { // make_tag —— 默认“区间加”语义
        /* 【在此更新下递函数】（需要“区间赋值/推平”时整段替换，见后文注释） */
        p.w += (p.r - p.l + 1) * lazy;
        p.rmq += lazy;
        p.lazy += lazy;
        p.tg = true;
    }
    void pushdown(int k) {
        if (!t[k].tg) return;
        pushdown(t[GL], t[k].lazy);
        pushdown(t[GR], t[k].lazy);
        t[k].lazy = 0;
        t[k].tg = false;
    }
    void pushup(int k) {
        auto pushup = [&](node &p, node &l, node &r) {
            /* 【在此更新上传函数】 */
            p.w = l.w + r.w;
            p.rmq = min(l.rmq, r.rmq); // RMQ -> min/max
        };
        pushup(t[k], t[GL], t[GR]);
    }
    void modify(int l, int r, T val, int k = 1) { // 区间修改
        if (l <= t[k].l && t[k].r <= r) {
```

```
            pushdown(t[k], val);   // 全覆盖：统一走打标入口，w/rmq/lazy/tg 一次更新到位
            return;
        }
        pushdown(k);
        int mid = (t[k].l + t[k].r) / 2;
        if (l <= mid) modify(l, r, val, GL);
        if (mid < r) modify(l, r, val, GR);
        pushup(k);
    }
    T rmq(int l, int r, int k = 1) { // 区间询问最小值
        if (l <= t[k].l && t[k].r <= r) {
            return t[k].rmq;
        }
        pushdown(k);
        int mid = (t[k].l + t[k].r) / 2;
        T ans = numeric_limits<T>::max(); // RMQ -> 为 max 时需要修改为 ::lowest()
        if (l <= mid) ans = min(ans, rmq(l, r, GL)); // RMQ -> min/max
        if (mid < r) ans = min(ans, rmq(l, r, GR)); // RMQ -> min/max
        return ans;
    }
    T ask(int l, int r, int k = 1) { // 区间询问
        if (l <= t[k].l && t[k].r <= r) {
            return t[k].w;
        }
        pushdown(k);
        int mid = (t[k].l + t[k].r) / 2;
        T ans = 0;
        if (l <= mid) ans += ask(l, r, GL);
        if (mid < r) ans += ask(l, r, GR);
        return ans;
    }
    void debug(int k = 1) {
        cout << "[" << t[k].l << ", " << t[k].r << "]: ";
        cout << "w = " << t[k].w << ", ";
        cout << "Min = " << t[k].rmq << ", ";
        cout << "lazy = " << t[k].lazy << ", ";
        cout << "tg = " << t[k].tg << ", ";
        cout << endl;
        if (t[k].l == t[k].r) return;
        debug(GL), debug(GR);
    }
};
```

赋值（推平）语义——只换 `pushdown(node&, T)` 一处，`modify` 不用动：

```
void pushdown(node &p, T lazy) { // make_tag —— “区间赋值”语义
    p.w    = (p.r - p.l + 1) * lazy; // 段和整体覆盖为 len*lazy
    p.rmq  = lazy;                   // 整段相等，min/max 都是 lazy
    p.lazy = lazy;                   // 后赋值直接覆盖先赋值的标记
    p.tg   = true;
}
```

珂朵莉树（OD Tree）

区间赋值的数据结构都可以骗分，在数据随机的情况下，复杂度可以保证，时间复杂度：$\mathcal{O}(N \log\log N)$。

```
struct ODT // 珂朵莉树 [1, n]
{
    struct Node
    {
        int l, r;
        mutable int v;
        Node(int l, int r = -1, int v = 0) : l(l), r(r), v(v) {}
        bool operator<(const Node &o) const { return l < o.l; }
    };
    set<Node> s;
    ODT() { s.clear(); }
    ODT(int n)
    {
        s.clear();
        s.emplace(1, n, 0);
    }

    set<Node>::iterator split(int pos)
```



```
    {
        auto it = s.lower_bound({pos});
        if (it != s.end() && it->l == pos)
            return it;
        it--;
        auto [l, r, v] = *it;
        s.erase(it);
        s.emplace(l, pos - 1, v);
        return s.emplace(pos, r, v).first;
    }
    void assign(int l, int r, int x)
    {
        auto itr = split(r + 1), itl = split(l);
        s.erase(itl, itr);
        s.emplace(l, r, x);
    }
    void add(int l, int r, int x)
    {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; it++)
            it->v += x;
    }
    int kth(int l, int r, int k)
    {
        vector<pair<int, int>> a;
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; it++)
            a.push_back({it->v, it->r - it->l + 1});
        sort(ALL(a));
        for (auto [val, len] : a)
        {
            k -= len;
            if (k <= 0)
                return val;
        }
        return -1;
    }
    int powersum(int l, int r, int x, int mod)
    {
        auto itr = split(r + 1), itl = split(l);
        int ans = 0;
        for (auto it = itl; it != itr; it++)
        {
            int len = it->r - it->l + 1;
            int val = it->v % mod;
            int pow = 1;
            for (int b = x; b; b >>= 1, val = val * val % mod)
                if (b & 1)
                    pow = pow * val % mod;
            ans = (ans + pow * len) % mod;
        }
        return ans;
    }
};
```

主席树

单点加 区间查询

```
template <typename T, typename F, int NODES = 20000000>
struct PersistentSegmentTree
{
    using ll = long long;
    struct Node
    {
        T data;
        Node *l, *r;
        Node() {}
        Node(const T &_data) : data(_data), l(nullptr), r(nullptr) {}
    };

    Node *pool;
    int pid;
    ll N;
    const F f;
    const T ID;
```

```
    Node *nil;
    vector<Node *> roots;

    PersistentSegmentTree(const vector<T> &v, const F &f_, const T &ID_)
        : pid(0), f(f_), ID(ID_), nil(nullptr)
    {
        pool = new Node[NODES];
        nil = my_new(ID);
        nil->l = nil->r = nil;
        roots.reserve(262144);
        roots.push_back(build(v));
    }

    PersistentSegmentTree(const ll &N_, const F &f_, const T &ID_)
        : pid(0), N(N_), f(f_), ID(ID_), nil(nullptr)
    {
        pool = new Node[NODES];
        nil = my_new(ID);
        nil->l = nil->r = nil;
        roots.reserve(262144);
        roots.push_back(nil);
    }

    Node *my_new(const T &data)
    {
        pool[pid].data = data;
        pool[pid].l = pool[pid].r = nil;
        return &(pool[pid++]);
    }

    Node *merge(Node *l, Node *r)
    {
        pool[pid].data = f(l->data, r->data);
        pool[pid].l = l;
        pool[pid].r = r;
        return &(pool[pid++]);
    }

    Node *build(const vector<T> &v)
    {
        N = (ll)v.size();
        return build(0, (ll)v.size(), v);
    }

    Node *build(ll l, ll r, const vector<T> &v)
    {
        if (l + 1 == r)
            return my_new(v[l]);
        ll m = (l + r) >> 1;
        return merge(build(l, m, v), build(m, r, v));
    }

private:
    Node *update_(ll a, const T &x, Node *n, ll l, ll r)
    {
        if (l + 1 == r)
            return my_new(x);
        ll m = (l + r) >> 1;
        if (a < m)
            return merge(update_(a, x, n->l, l, m), n->r);
        return merge(n->l, update_(a, x, n->r, m, r));
    }
    Node *add_(ll a, const T &x, Node *n, ll l, ll r)
    {
        if (l + 1 == r)
            return my_new(f(x, n->data));
        ll m = (l + r) >> 1;
        if (a < m)
            return merge(add_(a, x, n->l, l, m), n->r);
        return merge(n->l, add_(a, x, n->r, m, r));
    }
    T query_(ll a, ll b, Node *n, ll l, ll r)
    {
        if (n == nil)
            return ID;
        if (r <= a or b <= l)
            return ID;
        if (a <= l and r <= b)
            return n->data;
        ll m = (l + r) >> 1;
```



```
        return f(query_(a, b, n->l, l, m), query_(a, b, n->r, m, r));
    }

public:
    Node *update(Node *n, ll k, const T &x)
    {
        Node *root = update_(k, x, n, 0, N);
        roots.push_back(root);
        return root;
    }
    Node *update(int t, ll k, const T &x)
    {
        Node *root = update_(k, x, roots[t], 0, N);
        roots.push_back(root);
        return root;
    }
    Node *update(ll k, const T &x)
    {
        Node *root = update_(k, x, roots.back(), 0, N);
        roots.push_back(root);
        return root;
    }

    Node *add(Node *n, ll k, const T &x)
    {
        Node *root = add_(k, x, n, 0, N);
        roots.push_back(root);
        return root;
    }
    Node *add(int t, ll k, const T &x)
    {
        Node *root = add_(k, x, roots[t], 0, N);
        roots.push_back(root);
        return root;
    }
    Node *add(ll k, const T &x)
    {
        Node *root = add_(k, x, roots.back(), 0, N);
        roots.push_back(root);
        return root;
    }

    T query(Node *n, ll a, ll b) { return query_(a, b, n, 0, N); }
    T query(int t, ll a, ll b) { return query_(a, b, roots[t], 0, N); }
    T query(ll a, ll b) { return query_(a, b, roots.back(), 0, N); }

    Node *new_tree() { return nil; }
};
/*
auto f = [](long x, long y){ return x + y; };//查询函数
PersistentSegmentTree<long, decltype(f)> seg(M, f, 0L);
*/
```

kth 主席树

```
struct PresidentTree {
    static constexpr int N = 2e5 + 10;
    int cntNodes, root[N];
    struct node {
        int l, r;
        int cnt;
    } tr[4 * N + 17 * N];
    void modify(int &u, int v, int l, int r, int x) {
        u = ++cntNodes;
        tr[u] = tr[v];
        tr[u].cnt++;
        if (l == r) return;
        int mid = (l + r) / 2;
        if (x <= mid)
            modify(tr[u].l, tr[v].l, l, mid, x);
        else
            modify(tr[u].r, tr[v].r, mid + 1, r, x);
    }
    int kth(int u, int v, int l, int r, int k) {
        if (l == r) return l;
        int res = tr[tr[v].l].cnt - tr[tr[u].l].cnt;
```

```
        int mid = (l + r) / 2;
        if (k <= res)
            return kth(tr[u].l, tr[v].l, l, mid, k);
        else
            return kth(tr[u].r, tr[v].r, mid + 1, r, k - res);
    }
};
```

串

Tire 字典树

```
struct Tire
{
    vector<vector<int>> ch;
    vector<int> cnt;
    int idx;
    map<char, int> mp;
    Tire(int n = 1000)
    {
        ch.assign(n, vector<int>(63));
        cnt.assign(n, 0);
        idx = 0;
        LL id = 0;
        for (char c = 'a'; c <= 'z'; c++)
            mp[c] = ++id;
        for (char c = 'A'; c <= 'Z'; c++)
            mp[c] = ++id;
        for (char c = '0'; c <= '9'; c++)
            mp[c] = ++id;
    }
    void insert(const string &s)
    {
        int u = 0;
        for (int i = 0; i < s.size(); i++)
        {
            int id = mp[s[i]];
            if (!ch[u][id])
                ch[u][id] = ++idx;
            u = ch[u][id];
        }
        cnt[u]++;
    }
    LL query(const string &s)
    {
        int u = 0;
        for (int i = 0; i < s.size(); i++)
        {
            int id = mp[s[i]];
            if (!ch[u][id])
                return 0;
            u = ch[u][id];
        }
        return cnt[u];
    }
    void clear()
    {
        for (int i = 0; i <= idx; i++)
        {
            cnt[i] = 0;
            for (int j = 0; j <= 62; j++)
                ch[i][j] = 0;
        }
        idx = 0;
    }
};
```

KMP

```
//0-idx
auto kmp = [&](const string &s, const string &t)
{
    int n = s.size(), m = t.size();
```



```
    vector<int> next(m), ans;
    for (int i = 1, len = 0; i < m;i++){
        while(len&&t[i]!=t[len])
            len = next[len-1];
        next[i] = len += t[i] == t[len];
    }
    for (int i = 0, j = 0; i < n;i++){
        while(j&&s[i]!=t[j])
            j = next[j-1];
        if(s[i]==t[j]&&++j==m){
            ans.push_back(i - m+1);
        }
    }
    return ans;
};

//1-idx
auto kmp = [&](string s, string t)
{
    int n = s.size(), m = t.size();
    vector<int> next(m + 1), ans;
    s = " " + s,t = " " + t;
    for (int i = 2, j = 0; i <= m;i++){
        while(j&&t[i]!=t[j+1])
            j = next[j];
        next[i] = j += t[i] == t[j + 1];
    }
    for (int i = 1, j = 0; i <= n;i++){
        while(j&&s[i]!=t[j+1])
            j = next[j];
        if(s[i]==t[j+1]&&++j==m)
            ans.push_back(i - m + 1);
    }
    return ans;
};
```

AC 自动机

```
// Trie+Kmp，多模式串匹配
struct ACAutomaton {
    static constexpr int N = 1e6 + 10;
    int ch[N][26], fail[N], cntNodes;
    int cnt[N];
    ACAutomaton() {
        cntNodes = 1;
    }
    void insert(string s) {
        int u = 1;
        for (auto c : s) {
            int &v = ch[u][c - 'a'];
            if (!v) v = ++cntNodes;
            u = v;
        }
        cnt[u]++;
    }
    void build() {
        fill(ch[0], ch[0] + 26, 1);
        queue<int> q;
        q.push(1);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int i = 0; i < 26; i++) {
                int &v = ch[u][i];
                if (!v)
                    v = ch[fail[u]][i];
                else {
                    fail[v] = ch[fail[u]][i];
                    q.push(v);
                }
            }
        }
    }
    LL query(string t) {
        LL ans = 0;
        int u = 1;
        for (auto c : t) {
```

```
            u = ch[u][c - 'a'];
            for (int v = u; v && ~cnt[v]; v = fail[v]) {
                ans += cnt[v];
                cnt[v] = -1;
            }
        }
        return ans;
    }
};
```

图论

知识补充

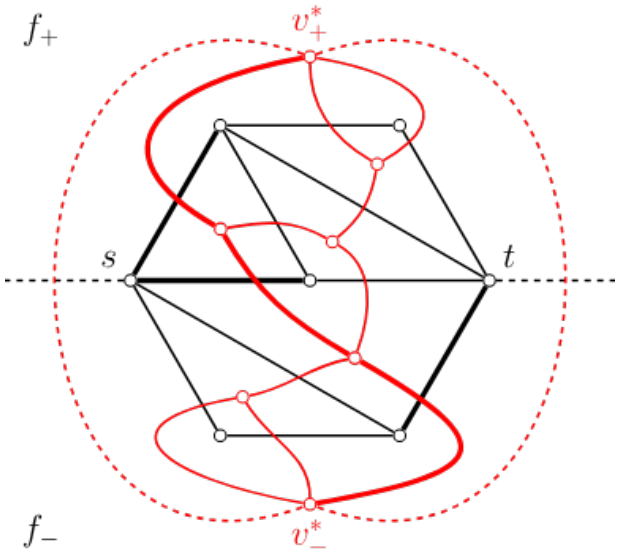
1. 树直径端点的判定树上任意一点能达到最远的点一定是树直径的端点
2. 环存在的充分条件：有向图中，所有点均有出度，则有环
3. **握手定理** 无向图中所有顶点的度数之和 $= 2|E|$；奇度顶点有偶数个
4. **二分图判定** 一个图是二分图 $\iff$ 不含奇环（可用染色法 $O(n)$ 判定，结合 DFS 树的返祖边可以找到该环）
5. **欧拉回路存在条件**（无向图）所有顶点度数为偶数；（有向图）所有顶点入度 $=$ 出度（前提：图连通 / 边在同一个弱连通分量中）
6. **竞赛图（Tournament）** n 个点的完全有向图
   - 一定存在 **哈密顿路径**
   - 一定存在 **哈密顿回路**（当且仅当图强连通时）
   - 缩点后是一条链（拓扑序唯一）
   - 若对每个出度求和：$\sum outdeg(v)^2 = \sum indeg(v)^2$
   - 出度序列与 Landau 定理相关，可用于判定一个序列是否为合法的竞赛图出度序列
7. **DFS 树**：无向图 dfs 树不会出现横叉边，只会出现返祖边，有向图则都可能出现。
8. **Landau 定理**：将出度序列 $d_1 \le d_2 \le \ldots \le d_n$ 排序后，存在竞赛图与之对应 $\iff$ 对每个 $k < n$ 有 $\sum_{i=1}^{k} d_i \ge \binom{k}{2}$，且 $k = n$ 时取等 $\sum_{i=1}^{n} d_i = \binom{n}{2}$。

平面图

- 欧拉公式：对于有 k 个连通分支的平面图 G，有

$$|V| - |E| + |F| = k + 1$$

结论 若存在一种平面嵌入使 s（起点）、t（终点）共面，则在该嵌入的对偶图中，f^+ 与 f^- 之间的最短路等于原图的 s-t 最小割。



常见子问题

判断图的连通性

常用于判断欧拉路径的存在

```
bool check(const vector<vector<int>> &g){
    vector<bool> st(g.size(), false);
    queue<int> q;
    st[1] = true;
    int cnt = 1;
    q.push(1);
    while (q.size())
    {
        int u = q.front();
        q.pop();
        for (int v : g[u])
        {
            if (!st[v])
            {
                st[v] = true;
                cnt++;
                q.push(v);
            }
        }
    }
    return cnt == g.size() - 1;
}
```

树论

树的直径

```
    vector<int> dis(n + 1); // 根到所有点的距离
    vector<int> fa(n + 1);  // dfs路径中所有点的前继
    int far=0;//每次dfs中根能到达的最远的点
    function<void(int, int)> dfs = [&](int u, int f) mutable -> void
    {
        fa[u] = f;
        if(dis[u]>dis[far])
```

```
            far = u;
        for(PII ne:g[u]){
            int v = ne.first, w = ne.second;
            if(v==f)
                continue;
            dis[v] = dis[u] + w;
            dfs(v, u);
        }
    };
    dis[1] = 0;
    dfs(1, 0);
    int A=far;
    dis[A]=0;
    dfs(A, 0);
    int B = far;
```

树的重心

```
    vector<int> f(n + 1);//f[u]保存u的最大子树的大小
    vector<int> size(n + 1);//size[u]表示u这个子树的大小
    int center = 0;//求解的重心
    f[0] = INF;
    function<void(int, int)> getcenter = [&](int u, int fa) mutable -> void
    {
        size[u] = 1, f[u] = 0;
        for (int v:g[u])
        {
            if(v==fa)
                continue;
            getcenter(v, u);
            size[u]+=size[v];
            f[u] = max(f[u], size[v]);
        }
        f[u] = max(f[u], n - size[u]);
        if (f[u] < f[center] || f[u] == f[center] && u < center)
            center = u;
    };
    getcenter(1,0);
```

最近公共祖先 LCA

5.3.3.1 树链剖分解法 预处理时间复杂度 $\mathcal{O}(N)$；单次查询 $\mathcal{O}(\log N)$，常数较小。

```
struct HLD {
    int n, idx;
    vector<vector<int>> ver;
    vector<int> siz, dep;
    vector<int> top, son, parent;

    HLD(int n) {
        this->n = n;
        ver.resize(n + 1);
        siz.resize(n + 1);
        dep.resize(n + 1);

        top.resize(n + 1);
        son.resize(n + 1);
        parent.resize(n + 1);
    }
    void add(int x, int y) { // 建立双向边
        ver[x].push_back(y);
        ver[y].push_back(x);
    }
    void dfs1(int x) {
        siz[x] = 1;
        dep[x] = dep[parent[x]] + 1;
        for (auto y : ver[x]) {
            if (y == parent[x]) continue;
            parent[y] = x;
            dfs1(y);
            siz[x] += siz[y];
            if (siz[y] > siz[son[x]]) {
                son[x] = y;
```



```
            }
        }
    }
    void dfs2(int x, int up) {
        top[x] = up;
        if (son[x]) dfs2(son[x], up);
        for (auto y : ver[x]) {
            if (y == parent[x] || y == son[x]) continue;
            dfs2(y, y);
        }
    }
    int lca(int x, int y) {
        while (top[x] != top[y]) {
            if (dep[top[x]] > dep[top[y]]) {
                x = parent[top[x]];
            } else {
                y = parent[top[y]];
            }
        }
        return dep[x] < dep[y] ? x : y;
    }
    int calc(int x, int y) { // 查询两点间距离
        return dep[x] + dep[y] - 2 * dep[lca(x, y)];
    }
    void work(int root = 1) { // 在此初始化
        dfs1(root);
        dfs2(root, root);
    }
};
```

5.3.3.2 树上倍增解法 预处理时间复杂度 $\mathcal{O}(N \log N)$；单次查询 $\mathcal{O}(\log N)$，但是常数比树链剖分解法更大。

封装一：基础封装，针对无权图。

```
struct Tree {
    int n;
    vector<vector<int>> ver, val;
    vector<int> lg, dep;
    Tree(int n) {
        this->n = n;
        ver.resize(n + 1);
        val.resize(n + 1, vector<int>(30));
        lg.resize(n + 1);
        dep.resize(n + 1);
        for (int i = 1; i <= n; i++) { //预处理 log
            lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
        }
    }
    void add(int x, int y) { // 建立双向边
        ver[x].push_back(y);
        ver[y].push_back(x);
    }
    void dfs(int x, int fa) {
        val[x][0] = fa; // 储存 x 的父节点
        dep[x] = dep[fa] + 1;
        for (int i = 1; i <= lg[dep[x]]; i++) {
            val[x][i] = val[val[x][i - 1]][i - 1];
        }
        for (auto y : ver[x]) {
            if (y == fa) continue;
            dfs(y, x);
        }
    }
    int lca(int x, int y) {
        if (dep[x] < dep[y]) swap(x, y);
        while (dep[x] > dep[y]) {
            x = val[x][lg[dep[x] - dep[y]] - 1];
        }
        if (x == y) return x;
        for (int k = lg[dep[x]] - 1; k >= 0; k--) {
            if (val[x][k] == val[y][k]) continue;
            x = val[x][k];
            y = val[y][k];
        }
        return val[x][0];
    }
```

```
    int clac(int x, int y) { // 倍增查询两点间距离
        return dep[x] + dep[y] - 2 * dep[lca(x, y)];
    }
    void work(int root = 1) { // 在此初始化
        dfs(root, 0);
    }
};
```

封装二：扩展封装，针对有权图，支持“倍增查询两点路径上的最大边权”功能。

```
struct Tree {
    int n;
    vector<vector<int>> val, Max;
    vector<vector<pair<int, int>>> ver;
    vector<int> lg, dep;
    Tree(int n) {
        this->n = n;
        ver.resize(n + 1);
        val.resize(n + 1, vector<int>(30));
        Max.resize(n + 1, vector<int>(30));
        lg.resize(n + 1);
        dep.resize(n + 1);
        for (int i = 1; i <= n; i++) { //预处理 log
            lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
        }
    }
    void add(int x, int y, int w) { // 建立双向边
        ver[x].push_back({y, w});
        ver[y].push_back({x, w});
    }
    void dfs(int x, int fa) {
        val[x][0] = fa;
        dep[x] = dep[fa] + 1;
        for (int i = 1; i <= lg[dep[x]]; i++) {
            val[x][i] = val[val[x][i - 1]][i - 1];
            Max[x][i] = max(Max[x][i - 1], Max[val[x][i - 1]][i - 1]);
        }
        for (auto [y, w] : ver[x]) {
            if (y == fa) continue;
            Max[y][0] = w;
            dfs(y, x);
        }
    }
    int lca(int x, int y) {
        if (dep[x] < dep[y]) swap(x, y);
        while (dep[x] > dep[y]) {
            x = val[x][lg[dep[x] - dep[y]] - 1];
        }
        if (x == y) return x;
        for (int k = lg[dep[x]] - 1; k >= 0; k--) {
            if (val[x][k] == val[y][k]) continue;
            x = val[x][k];
            y = val[y][k];
        }
        return val[x][0];
    }
    int clac(int x, int y) { // 倍增查询两点间距离
        return dep[x] + dep[y] - 2 * dep[lca(x, y)];
    }
    int query(int x, int y) { // 倍增查询两点路径上的最大边权（带权图）
        auto get = [&](int x, int y) -> int {
            int ans = 0;
            if (x == y) return ans;
            for (int i = lg[dep[x]]; i >= 0; i--) {
                if (dep[val[x][i]] > dep[y]) {
                    ans = max(ans, Max[x][i]);
                    x = val[x][i];
                }
            }
            ans = max(ans, Max[x][0]);
            return ans;
        };
        int fa = lca(x, y);
        return max(get(x, fa), get(y, fa));
    }
    void work(int root = 1) { // 在此初始化
        dfs(root, 0);
```



```
    }
};
```

树链剖分/轻重链剖分

将线段树处理的部分分离，方便修改。支持链上查询/修改、子树查询/修改，建树时间复杂度 $\mathcal{O}(N \log N)$，单次查询时间复杂度 $\mathcal{O}(\log^2 N)$。

```
struct Segt {
    struct node {
        int l, r, w, lazy;
    };
    vector<int> w;
    vector<node> t;

    Segt() {}
    #define GL (k << 1)
    #define GR (k << 1 | 1)

    void init(vector<int> in) {
        int n = in.size() - 1;
        w.resize(n + 1);
        for (int i = 1; i <= n; i++) {
            w[i] = in[i];
        }
        t.resize(n * 4 + 1);
        auto build = [&](auto self, int l, int r, int k = 1) {
            if (l == r) {
                t[k] = {l, r, w[l], 0}; // 如果有赋值为 0 的操作，则懒标记必须要 -1
                return;
            }
            t[k] = {l, r};
            int mid = (l + r) / 2;
            self(self, l, mid, GL);
            self(self, mid + 1, r, GR);
            pushup(k);
        };
        build(build, 1, n);
    }
    void pushdown(node &p, int lazy) {
    /* 【在此更新下递函数】 */
        p.w += (p.r - p.l + 1) * lazy;
        p.lazy += lazy;
    }
    void pushdown(int k) { // 不需要动
        if (t[k].lazy == 0) return;
        pushdown(t[GL], t[k].lazy);
        pushdown(t[GR], t[k].lazy);
        t[k].lazy = 0;
    }
    void pushup(int k) { // 不需要动
        auto pushup = [&](node &p, node &l, node &r) {
        /* 【在此更新上传函数】 */
            p.w = l.w + r.w;
        };
        pushup(t[k], t[GL], t[GR]);
    }
    void modify(int l, int r, int val, int k = 1) {
        if (l <= t[k].l && t[k].r <= r) {
            pushdown(t[k], val);
            return;
        }
        pushdown(k);
        int mid = (t[k].l + t[k].r) / 2;
        if (l <= mid) modify(l, r, val, GL);
        if (mid < r) modify(l, r, val, GR);
        pushup(k);
    }
    int ask(int l, int r, int k = 1) {
        if (l <= t[k].l && t[k].r <= r) {
            return t[k].w;
        }
        pushdown(k);
        int mid = (t[k].l + t[k].r) / 2;
        int ans = 0;
        if (l <= mid) ans += ask(l, r, GL);
```

```
        if (mid < r) ans += ask(l, r, GR);
        return ans;
    }
};

struct HLD {
    int n, idx;
    vector<vector<int>> ver;
    vector<int> siz, dep;
    vector<int> top, son, parent;
    vector<int> in, id, val;
    Segt segt;

    HLD(int n) {
        this->n = n;
        ver.resize(n + 1);
        siz.resize(n + 1);
        dep.resize(n + 1);

        top.resize(n + 1);
        son.resize(n + 1);
        parent.resize(n + 1);

        idx = 0;
        in.resize(n + 1);
        id.resize(n + 1);
        val.resize(n + 1);
    }
    void add(int x, int y) { // 建立双向边
        ver[x].push_back(y);
        ver[y].push_back(x);
    }
    void dfs1(int x) {
        siz[x] = 1;
        dep[x] = dep[parent[x]] + 1;
        for (auto y : ver[x]) {
            if (y == parent[x]) continue;
            parent[y] = x;
            dfs1(y);
            siz[x] += siz[y];
            if (siz[y] > siz[son[x]]) {
                son[x] = y;
            }
        }
    }
    void dfs2(int x, int up) {
        id[x] = ++idx;
        val[idx] = in[x]; // 建立编号
        top[x] = up;
        if (son[x]) dfs2(son[x], up);
        for (auto y : ver[x]) {
            if (y == parent[x] || y == son[x]) continue;
            dfs2(y, y);
        }
    }
    void modify(int l, int r, int val) { // 链上修改
        while (top[l] != top[r]) {
            if (dep[top[l]] < dep[top[r]]) {
                swap(l, r);
            }
            segt.modify(id[top[l]], id[l], val);
            l = parent[top[l]];
        }
        if (dep[l] > dep[r]) {
            swap(l, r);
        }
        segt.modify(id[l], id[r], val);
    }
    void modify(int root, int val) { // 子树修改
        segt.modify(id[root], id[root] + siz[root] - 1, val);
    }
    int ask(int l, int r) { // 链上查询
        int ans = 0;
        while (top[l] != top[r]) {
            if (dep[top[l]] < dep[top[r]]) {
                swap(l, r);
            }
            ans += segt.ask(id[top[l]], id[l]);
            l = parent[top[l]];
```



```
        }
        if (dep[l] > dep[r]) {
            swap(l, r);
        }
        return ans + segt.ask(id[l], id[r]);
    }
    int ask(int root) { // 子树查询
        return segt.ask(id[root], id[root] + siz[root] - 1);
    }
    void work(auto in, int root = 1) { // 在此初始化
        assert(in.size() == n + 1);
        this->in = in;
        dfs1(root);
        dfs2(root, root);
        segt.init(val); // 建立线段树
    }
    void work(int root = 1) { // 在此初始化
        dfs1(root);
        dfs2(root, root);
        segt.init(val); // 建立线段树
    }
};
```

并查集（DSU）

```
// 警示后人：维护并查集权值时，如果要进行合并操作，一定要检查是否合并成功再去合并权值
struct DSU // 并查集范围[1,n]
{
    vector<int> f, sz, val;
    int bg = 1;
    DSU(int n) : f(n + 1), sz(n + 1, 1), val(n + 1, 0)
    {
        iota(f.begin(), f.end(), 0);
    }
    int find(int x)
    {
        return f[x] == x ? x : (f[x] = find(f[x]));
    }
    bool same(int x, int y)
    {
        x += 1 - bg;
        y += 1 - bg;
        return find(x) == find(y);
    }
    bool merge(int x, int y) // 把x合并到y的集合上
    {
        x += 1 - bg;
        y += 1 - bg;
        x = find(x), y = find(y);
        if (x == y)
            return false;
        val[y] += val[x];//权重合并方式
        sz[y] += sz[x];
        f[x] = y;
        return true;
    }
    int addval(int x, int w) // 修改连通块权重
    {
        return val[find(x - bg + 1)] += w;
    }
    int getval(int x) // 获得连通块权重
    {
        return val[find(x - bg + 1)];
    }
    int cnt() // 计算连通块的数量O(n)
    {
        int res = 0;
        for (int i = 1; i < f.size(); i++)
            res += (f[i] == i);
        return res;
    }
    int size(int x) // 计算连通块的大小
    {
        return sz[find(x - bg + 1)];
    }
```

```
};
```

拓扑排序

是 DAG<——> 有拓扑排序 常用于判断一个图是不是 DAG

```
vector<int> topsort(){//如果返回的数组大小不等于点的个数，说明不存在拓扑排序/该图有环/该图不是DAG
    vector<int>res;
    queue<int>Q;
    for(int i=1;i<=n;i++){
        if(!ind[i])Q.push(i);
    }
    while(!Q.empty()){
        int u=Q.front();
        Q.pop();
        res.push_back(u);
        for(auto v:g[u]){
            ind[v]--;
            if(!ind[v])Q.push(v);
        }
    }
    return res;
}
```

最短路

单源最短路

二分约束最短路 节点状态最短路 由对偶图最短路求平面图最小割

5.6.1.1 (正权稀疏图) 堆优化 dijkstra $O(mlogn)$

```
function<vector<int>(int)> dijkstra = [&](int beg) mutable -> vector<int>{ // 返回以beg为起点的单源最短路
    vector<int> dis(n + 1, INT_MAX);
    vector<bool> st(n + 1);
    priority_queue<PII, vector<PII>, greater<PII>> q;
    q.emplace(dis[beg] = 0, beg);
    while (q.size()){
        int u = q.top().second, d = q.top().first;
        q.pop();
        if (st[u])
            continue;
        st[u] = true;
        for (auto [v,w] : g[u]){
            /*在这里加个if可以限制路径走向*/
            if (dis[v] > d + w)
                q.emplace(dis[v]=d + w, v);
        }
    }
    return dis;
};
```

次短路

```
function<vector<int>(int)> dijkstra = [&](int beg) mutable -> vector<int>{
    vector<int> dis(n + 1, INT_MAX),dis2(n+1,INT_MAX);
    dis[beg] = 0;
    priority_queue<PII, vector<PII>, greater<PII>> q;
    q.push({0, beg});
    while (q.size()){
        int u = q.top().second, d = q.top().first;
        q.pop();
        if(d>dis2[u])
            continue;
        for (PII ne : g[u]){
            int v = ne.first, w = ne.second;
            if (dis[v] > d + w){
                dis2[v] = dis[v];
                dis[v] = d + w;
                q.push({dis[v], v});
```



```
            }
            if(dis2[v]>d+w&&dis[v]<d+w){
                dis2[v]=d+w;
                q.push({dis2[v], v});
            }
        }
    }
    return dis2;
};
```

5.6.1.2 （负权图、判负环）Bellman-Ford $O(nm)$

```
int n, m, s;
cin >> n >> m >> s;

vector<tuple<int, int, i64>> ver(m + 1);
for (int i = 1; i <= m; ++i) {
    int x, y;
    i64 w;
    cin >> x >> y >> w;
    ver[i] = {x, y, w};
}

vector<i64> dis(n + 1, inf), chk(n + 1);
dis[s] = 0;
for (int i = 1; i <= 2 * n; ++i) { // 双倍松弛，获取负环信息
    vector<i64> backup = dis;
    for (int j = 1; j <= m; ++j) {
        auto [x, y, w] = ver[j];
        chk[y] |= (i > n && backup[x] + w < dis[y]);
        dis[y] = min(dis[y], backup[x] + w);
    }
}

for (int i = 1; i <= n; ++i) {
    if (i == s) {
        cout << 0 << " ";
    } else if (dis[i] >= inf / 2) {
        cout << "no ";
    } else if (chk[i]) {
        cout << "inf ";
    } else {
        cout << dis[i] << " ";
    }
}
```

5.6.1.3 （差分约束、负权图)SPFA $O(km)$(k 是常数，最坏情况下为 n，一般极小） 常用于差分约束：最短路径给出的是每个变量在可行解中可能的最小值；要得到最大值，通常需要转化为最长路径问题或做额外处理。

```
//g[u] = {v,w}
function<vector<int>(int)> SPFA = [&](int beg) mutable -> vector<int>{//返回dist数组，若无解(差分约束无解，存在负环)返回空数组
    vector<int> dist(n + 1, INT_MAX);
    vector<bool> st(n + 1, false); // 记录节点是否在队列中
    vector<int> cnt(n + 1,0);
    dist[beg] = 0, st[beg] = true; // 初始节点标记为在队列中
    queue<int> q;
    q.push(beg);
    cnt[beg]++;
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        st[u] = false; // 节点出队，标记为不在队列中
        for (auto &ne : g[u])
        {
            int v = ne.first, w = ne.second;
            if (dist[v] > dist[u] + w)
            { // 松弛操作
                dist[v] = dist[u] + w;
                if (!st[v])
                { // 如果v不在队列中
                    q.push(v);
```

```
                    st[v] = true; // 加入队列后标记为true
                    cnt[v]++;
                    if(cnt[v]>=n)//入队>=n次说明有负环
                        return vector<int>();
                }
            }
        }
    }
    return dist;
};
```

多源最短路

5.6.2.1 Floyed $O(n^3)$

```
//邻接矩阵存图
vector<vector<int>> dis(n + 1, vector<int>(n + 1, 0x3f3f3f3f));
for(int i=1;i<=n;i++)//初始化
    dis[i][i] = 0;
//Floyed核心部分
for(int k=1;k<=n;k++){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            dis[i][j] = min(dis[i][j], dis[i][k]+ dis[k][j]);
        }
    }
}
```

传递闭包

```
for(int k=1;k<=n;k++){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            b[i][j] = b[i][j] | (b[i][k] & b[k][j]);
        }
    }
}
```

5.6.2.2 （负权稀疏图全源最短路）Johnson $O(nm + nmlogn)$

```
    int n, m;
    cin >> n >> m;
    vector<vector<PII>> g(n + 1); // g[u] = {v, w}
    for (int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].emplace_back(v, w);
    }

    // 1. 势函数 h[v]：全初始化成 0，等价于「虚拟源向所有点连 0 权边」，不用真建这个点
    vector<int> h(n + 1, 0);
    for (int i = 1; i <= n; i++)
    {
        bool upd = false;
        for (int u = 1; u <= n; u++)
        {
            for (auto [v, w] : g[u])
            {
                if (h[u] + w < h[v])
                {
                    h[v] = h[u] + w;
                    upd = true;
                }
            }
        }
        if (!upd) // 收敛
            break;
        if (i == n) // 第 n 轮还能松弛 -> 有负环
        {
            cout << "Negative Cycle" << endl;
            return;
```



```
        }
    }

    // 2. 重赋权 w'(u,v) = w + h[u] - h[v]，由三角不等式保证 >= 0
    vector<vector<PII>> g2(n + 1);
    for (int u = 1; u <= n; u++)
    {
        for (auto [v, w] : g[u])
        {
            g2[u].emplace_back(v, w + h[u] - h[v]);
        }
    }

    // 3. 在非负权图上跑 n 次 Dijkstra
    vector<vector<int>> dis(n + 1, vector<int>(n + 1, INF));
    for (int s = 1; s <= n; s++)
    {
        vector<int> d(n + 1, INF);
        vector<bool> st(n + 1, false);
        PQ_<PII> pq;
        pq.emplace(d[s] = 0, s);
        while (pq.size())
        {
            auto [du, u] = pq.top();
            pq.pop();
            if (st[u])
                continue;
            st[u] = 1;
            for (auto [v, w] : g2[u])
            {
                if (d[v] > du + w)
                    pq.emplace(d[v] = du + w, v);
            }
        }
        // 4. 还原 d(s,t) = d'(s,t) - h[s] + h[t]
        for (int t = 1; t <= n; t++)
        {
            if (d[t] < INF) // 不可达的点别碰，否则 INF 会被减出脏值
                dis[s][t] = d[t] - h[s] + h[t];
        }
    }

    for (int s = 1; s <= n; s++)
    {
        for (int t = 1; t <= n; t++)
        {
            cout << dis[s][t] << " \n"[t == n];
        }
    }
```

顺带：这套「势函数 + 重赋权」也是 Dijkstra 版最小费用流（Primal-Dual）的原理，见下方费用流一节。

DAG 中最长路

```
//动态规划解法（拓扑排序解法）
dist[v]=max(dist[v],dist[u]+w); //u是前驱节点
```

最小生成树

朴素 Prim $O(n^2)$

```
function<int()> prim = [&]() mutable -> int
{
    vector<int> dist(n + 1, INT_MAX);
    vector<bool> st(n + 1, false);
    dist[1] = 0;
    int res = 0, tot = 0;
    while (true)
    {
        int u = 0;
        for (int i = 1; i <= n; i++)
            if (!st[i] && dist[i] < dist[u])
```

```
                u = i;
        if (!u)
            break;
        tot++;
        st[u] = true;
        res += dist[u];
        for (auto [v, w] : g[u])
            dist[v] = min(dist[v], w);
    }
    return (tot == n ? res : -1);
};
```

稀疏图 堆优化 Prim $O(mlogn)$

```
function<int()> prim = [&]() mutable -> int
{
    priority_queue<PII,vector<PII>,greater<PII>> q;
    vector<int>dist(n+1,INT_MAX);
    vector<bool> st(n + 1);
    int res = 0,tot=0;
    dist[1]=0,q.push({0, 1});
    while(q.size()){
        int u = q.top().second;
        q.pop();
        if(st[u])continue;
        st[u] = true, res += dist[u], tot++;
        for(auto[v,w]:g[u]){
            if(dist[v]>w){
                q.push({dist[v]=w, v});
            }
        }
    }
    return (tot==n?res:-1);
};
```

稠密图 kruskal $O(mlogm)$

kruskal 重构树常用于 瓶颈路径问题（如最大边权的最小值）以及连通性相关问题 在 kruskal 重构树上倍增 LCA 可以维护路径信息

```
function<int()> kruskal = [&]() mutable -> int{
    int res = 0,tot=1;
    DSU dsu(n);
    sort(edge.begin(),edge.end());
    for(auto[w,u,v]:edge)
    {
        if(dsu.same(u,v))
            continue;
        res += w;
        dsu.merge(u, v);
        tot++;
    }
    return tot == n ? res : -1;
};
```

二分图

概念 二分图：一个图能被分为左右两部分，任何一条边的两个端点都不在同一部分中。匹配（独立边集）：一个边的集合，这些边没有公共顶点。

性质 二分图最小点覆盖 = 最大匹配 最大独立集 = n - 最大匹配

染色法判定二分图

```
vector<int> col(n + 1);
int cnt1, cnt2;//维护二分图两个子集的大小
function<bool(int, int)> dfs = [&](int u, int c) -> bool
{
```



```
    col[u] = c, cnt1 += c == 1, cnt2 += c == 2;
    bool res = true;
    for (int v : g[u])
    {
        if (!col[v])
            res&=dfs(v,3-c);
        else if (col[v] == c)
            res = 0;
    }
    return res;
};
int ans = 0;
for (int i = 1; i <= n; i++)
{
    if (!col[i])
    {
        cnt1 = cnt2 = 0;
        if (dfs(i, 1))
            ans += max(cnt1, cnt2);
    }
}
```

二分图的最大匹配

5.8.2.1 匈牙利算法（KM 算法）$O(NM)$

```
const int N = 510, M = 100010;
int n1, n2, m;
vector<int> g[N];
int match[N]; // 右边的点匹配的点
bool st[N];   // 判重
bool find(int x){
    for (int i = 0,sz=g[x].size(); i<sz; i++){ // 遍历能匹配的所有点
        int j = g[x][i];
        if (!st[j]){// 如果这个点没有找过
            st[j] = true;// 标记为找过
            if (match[j] == 0 || find(match[j])) {
                match[j] = x;
                return true;
            }
        }
    }
    return false;
}
int main(){
    cin >> n1 >> n2 >> m;
    while (m--){
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }
    int res = 0;
    for (int i = 1; i <= n1; i++){
        memset(st, false, sizeof st);
        if (find(i))
            res++;
    }
    cout << res;
    return 0;
}
```

欧拉回路/欧拉路径

(暂空)

连通性问题

边双连通分量/缩点

```
/*
方法
tarjan tj(g) //用g这个图初始化tarjan算法类
tj.init(g) //重新初始化
tj.getBridgeTree() //获得边双连通分量缩点图
*/
struct tarjan{
    vector<vector<int>> g,ebcc;
    stack<int>stk;
    vector<int> dfn, low,bel;
    int tim;
    tarjan(vector<vector<int>>& g){
        init(g);
    }
    void init(vector<vector<int>>& g){
        this->g=g;
        dfn.assign(g.size(), 0);
        low.assign(g.size(), 0);
        bel.assign(g.size(), 0);
        ebcc.clear();
        while(stk.size())
            stk.pop();
        tim = 0;
        for(int i=1;i<g.size();i++){
            if(!dfn[i])
                dfs(i, -1);
        }
    }
    void dfs(int u,int fa){
        dfn[u]=low[u]=++tim;
        stk.push(u);
        int cnt=0;//重边计数器

        for(int v:g[u]){
            if(v==fa && ++cnt==1)
                continue;
            if(!dfn[v]){//未访问过的节点
                dfs(v, u);
                low[u]=min(low[u],low[v]);
                if(low[v]>dfn[u]){//发现割边/桥u--v,说明v子树是一个边双连通分量  如果需要，可以在这里统计割边
                    vector<int>temp;
                    while(true){
                        int x=stk.top();
                        stk.pop();
                        temp.push_back(x);
                        bel[x] = ebcc.size();
                        if(x==v)
                            break;
                    }
                    ebcc.push_back(temp);
                }
            }
            else{
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(fa==-1){
            vector<int> temp;
            while (true)
            {
                int x = stk.top();
                stk.pop();
                temp.push_back(x);
                bel[x] = ebcc.size();
                if (x == u)
                    break;
            }
            ebcc.push_back(temp);
        }
    }
    vector<vector<int>> getBrigeTree(){
        if(ebcc.empty())return{};
        vector<set<int>> tree(ebcc.size());
        for(int u=1;u<g.size();u++){
```



```
            for(int v:g[u]){
                if(bel[u]!=bel[v]){
                    tree[bel[u]].insert(bel[v]);
                    tree[bel[v]].insert(bel[u]);
                }
            }
        }
        vector<vector<int>>res;
        for (set<int> e:tree){
            res .push_back(vector<int>(e.begin(),e.end()));
        }
        return res;
    }
};
```

强连通分量/缩点

```
//两种初始化方式
// tarjan tj(g) 无点权tarjan
// tarjan tj(g,val) 带点权tarjan
class tarjan{
    int n;
    vector<vector<int>> g,sccs;
    vector<int> dfn, low, id, val, sccs_val;
    vector<bool> ins;
    stack<int> stk;
    int tim;
    bool use_val;

    void dfs(int u){
        dfn[u] = low[u] = ++tim;
        stk.push(u);
        ins[u] = true;
        for (int v : g[u]){
            if (!dfn[v]){
                dfs(v);
                low[u] = min(low[u], low[v]);
            }
            else if (ins[v]){
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]){
            vector<int> scc;
            int scc_val = 0;
            while (true){
                int v = stk.top();
                stk.pop();
                ins[v] = false;
                scc.push_back(v);
                id[v] = sccs.size(); // 记录SCC编号
                // 点权处理（仅在启用时累加）
                if (use_val)
                {
                    scc_val += val[v];
                }
                if (v == u)
                    break;
            }
            sccs.push_back(scc);
            if (use_val)
                sccs_val.push_back(scc_val);
        }
    }
public:
    tarjan(vector<vector<int>> _g, vector<int> _val = vector<int>()) : n(_g.size() - 1), g(_g), dfn(n + 1,
    0), low(n + 1, 0), ins(n + 1, false), tim(0), id(n + 1, -1), val(_val), use_val(!_val.empty())
    {
        for (int i = 1; i <= n; i++)
            if (!dfn[i])
                dfs(i);
    }
    vector<vector<int>> getSccs(){//获得强连通点集
        return sccs;
    }
    vector<int> getSccsVal(){//新图的val
```

```
        return sccs_val;
    }
    vector<int> getId(){//每个节点所属的scc编号
        return id;
    }
    vector<vector<int>> getDag(){//获得强连通缩点图  注意! 这里建图的下标是从0开始的{
        if (sccs.empty())
            return {};
        vector<set<int>> temp(sccs.size());
        for (int u = 1; u <= n; u++){
            for (int v : g[u])
            {
                if (id[u] != id[v])
                { // 避免自环
                    temp[id[u]].insert(id[v]);
                }
            }
        }
        vector<vector<int>> dag;
        dag.reserve(temp.size());
        for (auto &s : temp){
            dag.emplace_back(s.begin(), s.end());
        }
        return dag;
    }
};
```

网络流

- **最大流最小割定理**
  - st 割：将点集 V 分为 S 和 T，其中 $s \in S$，$t \in T$
  - 割的容量：从 S 到 T 的边容量之和
  - 最大流 $=$ 最小割：任意图的最大流值等于最小割容量
- **常见建图技巧**
  - 拆点：点有容量限制时，拆成 $v_{in} \to v_{out}$，中间边容量为点容量
  - 分层图：按时间/步骤分层，层间连边，用于调度类问题
  - 二分图最大匹配转流：源点 $\to$ 左部 (1) $\to$ 右部 (1) $\to$ 汇点
  - 最小路径覆盖（DAG）：拆点后建二分图，最小路径覆盖 $= n -$ 最大匹配
  - 最大权闭合子图：正权点连源点，负权点连汇点，跑最小割，正权和 $-$ 最小割

最大流

5.11.1.1 Dinic $O(N^2M)$

```
template <typename T>
struct Flow_
{
    const int n;
    const T inf = numeric_limits<T> :: max();
    struct Edge
    {
        int to;
        T w;
        Edge(int to, T w) : to(to), w(w) {}
    };
    vector<Edge> ver;
    vector<vector<int>> h;
    vector<int> cur, d;
    Flow_(int n) : n(n + 1), h(n + 1) {}
    void add(int u, int v, T c)
    {
        h[u].push_back(ver.size());
        ver.emplace_back(v, c);
        h[v].push_back(ver.size());
        ver.emplace_back(u, 0);
    }
    bool bfs(int s, int t)
    {
        d.assign(n, -1);
        d[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty())
```



```
        {
            auto x = q.front();
            q.pop();
            for (auto it : h[x])
            {
                auto [y, w] = ver[it];
                if (w && d[y] == -1)
                {
                    d[y] = d[x] + 1;
                    if (y == t)
                        return true;
                    q.push(y);
                }
            }
        }
        return false;
    }
    T dfs(int u, int t, T f)
    {
        if (u == t)
            return f;
        auto r = f;
        for (int &i = cur[u]; i < h[u].size(); i++)
        {
            auto j = h[u][i];
            auto &[v, c] = ver[j];
            auto &[u, rc] = ver[j ^ 1];
            if (c && d[v] == d[u] + 1)
            {
                auto a = dfs(v, t, std::min(r, c));
                c -= a;
                rc += a;
                r -= a;
                if (!r)
                    return f;
            }
        }
        return f - r;
    }
    T work(int s, int t)
    {
        T ans = 0;
        while (bfs(s, t))
        {
            cur.assign(n, 0);
            ans += dfs(s, t, inf);
        }
        return ans;
    }
};
using Flow = Flow_<int>;
```

最小割

基础模型：构筑二分图，左半部 n 个点代表盈利项目，右半部 m 个点代表材料成本，收益为盈利之和减去成本之和，求最大收益。

建图：建立源点 S 向左半部连边，建立汇点 T 向右半部连边，如果某个项目需要某个材料，则新增一条容量 +∞ 的跨部边。

割边：放弃某个项目则断开 S 至该项目的边，购买某个原料则断开该原料至 T 的边，最终的图一定不存在从 S 到 T 的路径，此时我们得到二分图的一个 S–T 割。此时最小割即为求解最大流，边权之和减去最大流即为最大收益。

```
signed main()
{
    int n, m;
    cin >> n >> m;
    int S = n + m + 1, T = n + m + 2;
    Flow flow(T);
    for (int i = 1; i <= n; i++)
    {
        int w;
        cin >> w;
        flow.add(S, i, w);
    }
    int sum = 0;
    for (int i = 1; i <= m; i++)
```

```
    {
        int x, y, w;
        cin >> x >> y >> w;
        flow.add(x, n + i, 1E18);
        flow.add(y, n + i, 1E18);
        flow.add(n + i, T, w);
        sum += w;
    }
    cout << sum - flow.work(S, T) << endl;
}
```

费用流

5.11.3.1 Primal-Dual 原始对偶算法 $O(NMf)$，其中 f 代表最大流

```
struct MinCostFlow {
    using LL = long long;
    using PII = pair<LL, int>;
    const LL INF = numeric_limits<LL>::max();
    struct Edge {
        int v, c, f;
        Edge(int v, int c, int f) : v(v), c(c), f(f) {}
    };
    const int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<LL> h, dis;
    vector<int> pre;

    MinCostFlow(int n) : n(n), g(n) {}
    void add(int u, int v, int c, int f) { // c 流量, f 费用
        // if (f < 0) {
        //     g[u].push_back(e.size());
        //     e.emplace_back(v, 0, f);
        //     g[v].push_back(e.size());
        //     e.emplace_back(u, c, -f);
        // } else {
            g[u].push_back(e.size());
            e.emplace_back(v, c, f);
            g[v].push_back(e.size());
            e.emplace_back(u, 0, -f);
        // }
    }
    bool dijkstra(int s, int t) {
        dis.assign(n, INF);
        pre.assign(n, -1);
        priority_queue<PII, vector<PII>, greater<PII>> que;
        dis[s] = 0;
        que.emplace(0, s);
        while (!que.empty()) {
            auto [d, u] = que.top();
            que.pop();
            if (dis[u] < d) continue;
            for (int i : g[u]) {
                auto [v, c, f] = e[i];
                if (c > 0 && dis[v] > d + h[u] - h[v] + f) {
                    dis[v] = d + h[u] - h[v] + f;
                    pre[v] = i;
                    que.emplace(dis[v], v);
                }
            }
        }
        return dis[t] != INF;
    }
    pair<int, LL> flow(int s, int t) {
        int flow = 0;
        LL cost = 0;
        h.assign(n, 0);
        while (dijkstra(s, t)) {
            for (int i = 0; i < n; ++i) h[i] += dis[i];
            int aug = numeric_limits<int>::max();
            for (int i = t; i != s; i = e[pre[i] ^ 1].v) aug = min(aug, e[pre[i]].c);
            for (int i = t; i != s; i = e[pre[i] ^ 1].v) {
                e[pre[i]].c -= aug;
                e[pre[i] ^ 1].c += aug;
            }
            flow += aug;
            cost += LL(aug) * h[t];
        }
        return {flow, cost};
    }
};
```


经典动态规划

优化方法

方法	识别特征	复杂度改善
滚动数组	只依赖上一层	$O(nm) \to O(m)$ 空间
前缀和	转移是区间求和	$O(n^2) \to O(n)$
单调队列	滑动窗口最值	$O(nk) \to O(n)$
斜率优化	$i \cdot j$ 交叉项	$O(n^2) \to O(n)$
决策单调性	四边形不等式	$O(n^2) \to O(n \log n)$
线段树/BIT	任意区间转移	$O(n^2) \to O(n \log n)$
状压	状态是集合	指数级状态可控
矩阵快速幂	线性递推 + 大 n	$O(n) \to O(\log n)$
CDQ 分治	转移有偏序	$O(n^2) \to O(n \log^2 n)$

背包问题

01 背包

```
const int N = 1010;
int n, m;
int v[N], w[N];
int f[N];
int main(){
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        cin >> v[i] >> w[i];
    for (int i = 1; i <= n; i++)
        for (int j = m; j >= v[i]; j--)
            f[j] = max(f[j], f[j - v[i]] + w[i]);
    cout << f[m];
    return 0;
}
```

完全背包

```
const int N = 1010;
int n,m;
int v[N], w[N];
int f[N];
int main(){
    cin>>n>>m;
    for (int i = 1;i<=n;i++)cin>>v[i]>>w[i];
    for(int i=1;i<=n;i++)
        for (int j = v[i]; j <= m;j++)
            f[j] = max(f[j], f[j - v[i]] + w[i]);
    cout << f[m];
    return 0;
}
```

二进制优化多重背包

```
inline void solve(){
    int n = rd, m = rd;
    int cnt = 0;
    for (int i = 1; i <= n; i++){
        int a=rd,b=rd,s=rd;
        int k = 1;
        while(k<=s){
            cnt++;
            v[cnt] = a * k;
            w[cnt] = b * k;
            s -= k;
            k *= 2;
        }
        if(s>0){
            cnt++;
            v[cnt] = a * s;
            w[cnt] = b * s;
        }
    }
    n = cnt;
    for(int i=1;i<=n;i++)
        for(int j=m;j>=v[i];j--)
            f[j] = max(f[j], f[j - v[i]] + w[i]);
    cout << f[m];

}
```

分组背包

```
inline void solve()
{
    int n = rd, m = rd;
    for (int i = 1; i <= n; i++)
    {
        s[i] = rd;
        for (int j = 0; j < s[i]; j++)
            v[i][j] = rd, w[i][j] = rd;
    }
    for (int i = 1; i <= n; i++)
        for (int j = m; j >= 0; j--)
            for (int k = 0; k < s[i]; k++)
                if (v[i][k] <= j)
                    f[j] = max(f[j], f[j - v[i][k]] + w[i][k]);
    cout << f[m];
}
```

子序列问题

最长公共子序列

```
void solve()
{
    cin >> n >> m >> a >> b;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            f[i][j] = max(f[i - 1][j], f[i][j - 1]);
            if (a[i - 1] == b[j - 1])
                f[i][j] = max(f[i][j], f[i - 1][j - 1] + 1);
        }
    }
    cout << f[n][m];
}
```


最长上升子序列 LIS

- 最大上升子序列问题中的 Dilworth 定理 下降子序列的最大数量 = 不减子序列的最大长度 上升子序列的最大数量 = 不增子序列的最大长度

```
template <class T>
int LIS(const vector<T> &a)
{
    vector<T> tails;
    for (auto x : a)
    {
        auto it = upper_bound(ALL(tails), x, greater<T>());
        if (it == tails.end())
            tails.push_back(x);
        else
            *it = x;
    }
    return tails.size();
}
```

杂

库函数

- `lower_bound`：返回第一个 **不小于** `value` 的位置，即 `comp(*it, value) = false`。
- `upper_bound`：返回第一个 **大于** `value` 的位置，即 `comp(value, *it) = true`
- `sort`
  - 注意 sort 的 cmp 必须满足严格弱序
    - 反自反性`cmp(u,v) == false`
    - 传递性
    - 不可比的传递性若`a b 不可比,b c不可比，则cmp(a,b)==cmp(b,a) ==cmp(a,c)==false`

取模类

```
using i64 = long long;

template<class T> constexpr T mypow(T n, i64 k) {
    T r = 1;
    for (; k; k /= 2, n *= n) {
        if (k % 2) {
            r *= n;
        }
    }
    return r;
}

template<class T> constexpr T power(int n) {
    return mypow(T(2), n);
}

template<const int &MOD> struct Zmod {
    int x;
    Zmod(signed x = 0) : x(norm(x % MOD)) {}
    Zmod(i64 x) : x(norm(x % MOD)) {}

    constexpr int norm(int x) const noexcept {
        if (x < 0) [[unlikely]] {
            x += MOD;
        }
        if (x >= MOD) [[unlikely]] {
            x -= MOD;
        }
        return x;
    }
    explicit operator int() const {
        return x;
    }
    constexpr int val() const {
```

```
        return x;
    }
    constexpr Zmod operator-() const {
        Zmod val = norm(MOD - x);
        return val;
    }
    constexpr Zmod inv() const {
        assert(x != 0);
        return mypow(*this, MOD - 2);
    }
    friend constexpr auto &operator>>(istream &in, Zmod &j) {
        int v;
        in >> v;
        j = Zmod(v);
        return in;
    }
    friend constexpr auto &operator<<(ostream &o, const Zmod &j) {
        return o << j.val();
    }
    constexpr Zmod &operator++() {
        x = norm(x + 1);
        return *this;
    }
    constexpr Zmod &operator--() {
        x = norm(x - 1);
        return *this;
    }
    constexpr Zmod operator++(signed) {
        Zmod res = *this;
        ++*this;
        return res;
    }
    constexpr Zmod operator--(signed) {
        Zmod res = *this;
        --*this;
        return res;
    }
    constexpr Zmod &operator+=(const Zmod &i) {
        x = norm(x + i.x);
        return *this;
    }
    constexpr Zmod &operator-=(const Zmod &i) {
        x = norm(x - i.x);
        return *this;
    }
    constexpr Zmod &operator*=(const Zmod &i) {
        x = i64(x) * i.x % MOD;
        return *this;
    }
    constexpr Zmod &operator/=(const Zmod &i) {
        return *this *= i.inv();
    }
    constexpr Zmod &operator%=(const int &i) {
        return x %= i, *this;
    }
    friend constexpr Zmod operator+(const Zmod i, const Zmod j) {
        return Zmod(i) += j;
    }
    friend constexpr Zmod operator-(const Zmod i, const Zmod j) {
        return Zmod(i) -= j;
    }
    friend constexpr Zmod operator*(const Zmod i, const Zmod j) {
        return Zmod(i) *= j;
    }
    friend constexpr Zmod operator/(const Zmod i, const Zmod j) {
        return Zmod(i) /= j;
    }
    friend constexpr Zmod operator%(const Zmod i, const int j) {
        return Zmod(i) %= j;
    }
    friend constexpr bool operator==(const Zmod i, const Zmod j) {
        return i.val() == j.val();
    }
    friend constexpr bool operator!=(const Zmod i, const Zmod j) {
        return i.val() != j.val();
    }
    friend constexpr bool operator<(const Zmod i, const Zmod j) {
        return i.val() < j.val();
    }
    friend constexpr bool operator>(const Zmod i, const Zmod j) {
```



```
        return i.val() > j.val();
    }
};

int MOD[] = {998244353, 1000000007};
using Z = Zmod<MOD[1]>;
```

读入一行数组（个数未知）

```
string s;
getline(cin,s);
stringstream ss;
ss<<s;
while(ss>>s){
    auto res= stoi(s);
}
```

安全访问 mp

```
if (auto it = mp.find(key); it != mp.end()) {
    // 在这里使用 it->first 和 it->second
    // it 的作用域仅限于这个 if 块
}
```

vector<PII> 赋值 vector<int>

```
ranges::transform(a, back_inserter(b), &PII::second);
```

正则括号序列（RBS）

定义：$(\to +1,\ )\to -1$，合法序列 $\iff$ 全程 balance ≥ 0 且最终 $=0$。

- **最大合法子串长度**：栈存下标，遇到匹配的) 时弹栈，区间 `[stk.top()+1, i]` 为合法子串
- **最少添加括号数**：遍历中 balance 最低到达 $-k$，末尾 balance 为 b，则需要补 k 个 (和 $\max(0, b+k)$ 个)
- **嵌套深度**：遍历时 balance 的最大值即最大嵌套层数
- **交换相邻括号**：$)(\to ()$ 会使 balance $+2$，这是最少交换次数类问题的核心观察
- **字典序下一个合法序列**：从后往前找到第一个能改成 (的位置（需满足 balance 约束），改成 (后尽量填 (
- **本质不同合法括号序列计数**：n 对括号 $\Rightarrow$ Catalan 数 $C_n=\frac{1}{n+1}\binom{2n}{n}$

RBS 建树

对一棵有根树做 DFS，进入节点时写一个(，离开节点时写一个)，得到的序列就是合法括号序列；反之，任何合法括号序列都可以还原为一棵有序有根树。

```
auto RBSTREE = [&](string s)
{
    vector<vector<int>> g(s.size() / 2 + 1);
    stack<int> stk;
    stk.emplace(0); // 虚拟根节点0
    int cur = 0, cnt = 0;
    for (char c : s)
    {
        if (c == '(')
        {
            cur++;
            if (!stk.empty()) // 如果有父节点
            {
                int fa = stk.top(); // 父节点是上一级括号
                g[fa].emplace_back(cur);
            }
            stk.emplace(cur);
        }
```

```
        else
            stk.pop();
    }
    return g;
};
```

维护 mex

基础 mex

```
int mex(vector<int>& arr) {//O(nlogn)
    set<int> s(arr.begin(), arr.end());
    for(int i=0;i<=arr.size();i++)
        if(!s.contains(i))
            return i;
}
int mex(vector<int>& arr) {//O(n)
    vector<bool> occ(arr.size()+1);
    for(int x:arr)occ[x] = 1;
    for(int i=0;i<occ.size();i++)
        if(!occ[i]) return i;
}
```

动态 mex

也可以使用优先队列实现 内存开销不高时，cnt 可以用 vector 实现

```
class Mex
{
    map<int, int> cnt;
    set<int> missing;
    int size;

public:
    Mex(int _size = 100) : size(_size), missing(from_range, views::iota(0, _size + 1)) {};
    void insert(int x)
    {
        cnt[x]++;
        if (cnt[x] == 1)
            missing.erase(x);
    }

    void erase(int x)
    {
        if (--cnt[x] == 0)
        {
            missing.insert(x);
            cnt.erase(x);
        }
    }

    int query()
    {
        return *missing.begin();
    }
};
```

离线莫队 mex

```
struct Mex
{
    vector<int> arr, cnt, bcnt, ans; // cnt[x] = 值x在当前区间中的出现次数，bcnt[i] = 第i个值域块中缺失的数字个数（cnt=0的数字个数）
    vector<tuple<int, int, int>> qs;
    int n, block, val_block;

    Mex(const vector<int> &a) : arr(a)//初始数据a(0-based)
    {
        n = arr.size();
```



```
        int max_val = n + 2;
        val_block = sqrt(max_val) + 1;
        cnt.assign(max_val, 0);
        bcnt.assign((max_val + val_block - 1) / val_block, 0); // 初始化值域分块数组：共有 ceil(max_val / val_block) 个块

        // 初始化缺失计数
        for (int i = 0; i < bcnt.size(); i++)
        {
            int start = i * val_block;
            int end = min(max_val, start + val_block);
            bcnt[i] = end - start;
        }

        block = sqrt(n) + 1;
    }

    void add_q(int l, int r)
    {
        qs.emplace_back(l, r, qs.size());
    }

    void add(int idx)
    {
        int x = arr[idx];
        if (x < cnt.size() && ++cnt[x] == 1)
            bcnt[x / val_block]--;
    }

    void del(int idx)
    {
        int x = arr[idx];
        if (x < cnt.size() && --cnt[x] == 0)
            bcnt[x / val_block]++;
    }

    int mex()
    {
        int b = 0;
        while (b < bcnt.size() && bcnt[b] == 0)
            b++;
        if (b >= bcnt.size())
            return cnt.size();
        int start = b * val_block;
        int end = min((int)cnt.size(), start + val_block);
        for (int i = start; i < end; i++)
            if (cnt[i] == 0)
                return i;
        return cnt.size();
    }

    vector<int> solve()
    {
        if (qs.empty())
            return {};

        sort(qs.begin(), qs.end(), [&](const auto &a, const auto &b)
             {
            auto [l1, r1, i1] = a;
            auto [l2, r2, i2] = b;
            int b1 = l1 / block, b2 = l2 / block;
            if (b1 != b2) return b1 < b2;
            return (b1 & 1) ? (r1 > r2) : (r1 < r2); });//奇偶优化莫队。降低常数

        ans.resize(qs.size());
        int l = 0, r = -1;

        for (auto [ql, qr, idx] : qs)
        {
            while (l > ql)
                add(--l);
            while (r < qr)
                add(++r);
            while (l < ql)
                del(l++);
            while (r > qr)
                del(r--);
            ans[idx] = mex();
        }
```

```
        return ans;
    }
};
```